

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 13 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌──────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
164 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184
185 template <const int kWarpSize = WARP_SIZE, typename T = float>
186 __device__ __forceinline__ T warp_reduce_sum(T val) {
187 #pragma unroll
188   for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
189     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
190   }
191   return val;
192 }

```

```

189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
211 template <const int NUM_THREADS = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     __shared__ float shared[NUM_WARPS];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<NUM_WARPS>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int NUM_THREADS = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     __shared__ float shared[NUM_WARPS];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<NUM_WARPS>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;
251 // Grid: ((N + 255) / 256, 1, 1)

```

```

252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int NUM_THREADS = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * NUM_THREADS + tid;
258     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float shared[NUM_WARPS];
260
261     float val = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     val = warp_reduce_sum<WARP_SIZE>(val);
266     if (lane == 0)
267         shared[warp] = val;
268     __syncthreads();
269
270     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
271     if (warp == 0)
272         val = warp_reduce_sum<NUM_WARPS>(val);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
274         atomicAdd(y, val);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int NUM_THREADS = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * NUM_THREADS + tid;
286     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float shared[NUM_WARPS];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         shared[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<NUM_WARPS>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int NUM_THREADS = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
314     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

315     __shared__ float shared[NUM_WARPS];
316
317     float4 reg_a = FLOAT4(a[idx]);
318     float4 reg_b = FLOAT4(b[idx]);
319     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                     : 0.0f;
322     int warp = tid / WARP_SIZE;
323     int lane = tid % WARP_SIZE;
324
325     prod = warp_reduce_sum<WARP_SIZE>(prod);
326     if (lane == 0)
327         shared[warp] = prod;
328     __syncthreads();
329
330     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
331     if (warp == 0)
332         prod = warp_reduce_sum<NUM_WARPS>(prod);
333     if (tid == 0)
334         atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)
377 // Block: (256, 1, 1)

```

```

378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
420 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
421 //
422 // 面试要点:
423 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
424 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
425 //   - 数学公式 (5 步推导):
426 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
427 //         L_max = max(LSE_1, LSE_2)
428 //     Step 2 — 还原指数权重 (un-normalized):
429 //         w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
430 //     Step 3 — 归一化为混合比例:
431 //         alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
432 //         满足 alpha + beta = 1
433 //     Step 4 — 逐元素加权合并:
434 //         O = alpha * O_1 + beta * O_2
435 //   - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
436 //     lse[head_idx][token_idx])
437 //   - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
438 //     展平为 [T0H0, T0H1, ..., T1H0, ...])
439 //   - -inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
440 //     为 -inf) 的 LSE 可能为 +inf, 替换后 exp(-inf - L_max) = 0,

```

```

441 // 该段权重退化为 0
442 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
443 //   pack load → FMA → pack store 一条流水线
444 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
445 //
446 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
447 //   pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
448 //   threads_per_head = head_size / 4 = 2
449 //   total_threads = num_tokens * num_heads * threads_per_head = 8
450 //
451 //   global_idx:      0      1      2      3      4      5      6      7
452 //   token_head_idx:  0      0      1      1      2      2      3      3
453 //   (第几个 (token,head) 对, 行优先)
454 //   pack_idx:        0      1      0      1      0      1      0      1
455 //   (该 (token,head) 对内第几个 pack)
456 //   token_idx:        0      0      0      0      1      1      1      1
457 //   (token_head_idx / num_heads, token 变化慢)
458 //   head_idx:         0      0      1      1      0      0      1      1
459 //   (token_head_idx % num_heads, head 变化快)
460 //   pack_offset:      0      4      0      4      0      4      0      4
461 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
462 //   head_offset:      0      0      8      8     16     16     24     24
463 //   (token_idx * num_heads * head_size + head_idx * head_size,
464 //    该 (token,head) 对在 output 展平数组中的起始偏移)
465 //
466 // Grid: ((total_threads + 127) / 128, 1, 1)
467 // Block: (128, 1, 1)
468 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
469 _global__ void merge_attn_states(
470     float *output,           // [num_tokens, num_heads, head_size]
471     const float *prefix_output, // [num_tokens, num_heads, head_size]
472     const float *prefix_lse,   // [num_heads, num_tokens]
473     const float *suffix_output, // [num_tokens, num_heads, head_size]
474     const float *suffix_lse,   // [num_heads, num_tokens]
475     int num_tokens, int num_heads, int head_size) {
476
477     constexpr int NUM_THREADS = 128;
478     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
479     using pack_t = uint4;
480
481     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
482     const int threads_per_head = head_size / PACK_SIZE;
483     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
484     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
485     const int total_threads = num_tokens * num_heads * threads_per_head;
486
487     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
488     if (global_idx >= total_threads)
489         return;
490
491     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
492     // token_head_idx: 第几个 (token, head) 对, 行优先展平
493     const int token_head_idx = global_idx / threads_per_head;
494     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
495     const int pack_idx = global_idx % threads_per_head;
496
497     // token_head_idx 分解为 (token_idx, head_idx)
498     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
499     const int head_idx = token_head_idx % num_heads;   // head 变化快 (内维)
500
501     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
502     const int pack_offset = pack_idx * PACK_SIZE;
503     // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移

```

```

504 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
505
506 // 定位到当前 (token, head) 的输出段起始
507 const float *prefix_head = prefix_output + head_offset;
508 const float *suffix_head = suffix_output + head_offset;
509 float *output_head = output + head_offset;
510
511 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
512 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
513 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
514
515 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
516 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
517 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
518
519 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
520 const float max_lse = fmaxf(p_lse, s_lse);
521 p_lse -= max_lse;
522 s_lse -= max_lse;
523
524 // Step 2-3: 指数还原 → 混合比例 alpha, beta
525 const float p_se = expf(p_lse);
526 const float s_se = expf(s_lse);
527 const float out_se = p_se + s_se;
528 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
529 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
530
531 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
532 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
533 if (pack_offset < head_size) {
534     pack_t p_pack =
535         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
536     pack_t s_pack =
537         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
538     pack_t o_pack;
539
540 #pragma unroll
541     for (int i = 0; i < PACK_SIZE; ++i) {
542         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
543         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
544         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
545         reinterpret_cast<float *>(&o_pack)[i] = o_v;
546     }
547
548     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
549 }
550 }
551
552 // =====
553 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
554 // =====
555 // 面试要点:
556 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
557 //     online (1-pass, 增量更新)
558 //   - Online Softmax 是 FlashAttention 的数学基础
559 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
560 //     + variance)
561 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
562
563 // =====
564 // Phase 3a: Softmax — 三级递进 (面试核心考点)
565 // =====
566 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」

```

```

567 // 回答线索: naive(溢出) → safe → online
568
569 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
570 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
571 // 问题: x 值很大时 exp(x) 溢出为 inf
572 template <const int NUM_THREADS = 256>
573 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
574 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
575 // source: LeetCUDA/kernels/softmax/softmax.cu
576 __global__ void softmax_per_token(float *x, float *y, int N) {
577     const int tid = threadIdx.x;
578     const int idx = blockIdx.x * blockDim.x + tid;
579
580     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
581     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
582     if (idx < N)
583         y[idx] = exp_val / exp_sum;
584 }
585
586 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
587 // 面试重点: 为什么 Softmax 需要 Safe?
588 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
589 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
590 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
591 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
592 // Grid: (S, 1, 1)
593 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
594 // source: LeetCUDA/kernels/softmax/softmax.cu
595 template <const int NUM_THREADS = 256>
596 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
597     const int tid = threadIdx.x;
598     const int idx = blockIdx.x * blockDim.x + tid;
599
600     // Pass 1: block reduce max — 找最大值
601     float val = (idx < N) ? x[idx] : -FLT_MAX;
602     float max_val = block_reduce_max<NUM_THREADS>(val);
603
604     // Pass 2: exp(x - max) → block reduce sum
605     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
606     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
607
608     if (idx < N)
609         y[idx] = exp_val / exp_sum;
610 }
611
612 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
613 // 面试重点:
614 // 两种使用场景 (公式不同, 但结果等价):
615 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
616 //     m_new = max(m_old, x_i)
617 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
618 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
619 //     m = max(m1, m2) safe softmax用的max值
620 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
621 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
622 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
623 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
624 //
625 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
626 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
627 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
628 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2
629 //

```

```

630 // 合并公式 (对称, 不依赖"新/旧"概念):
631 //   m = max(m1, m2)
632 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
633 //
634 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
635 struct __align__(8) MD {
636     float m; // running max
637     float d; // running denominator (sum of exp(x - max))
638 };
639
640 template <const int kWarpSize = WARP_SIZE>
641 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
642     #pragma unroll
643     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
644         MD md2;
645         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
646         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
647
648         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
649         MD s_m = (md1.m > md2.m) ? md2 : md1;
650
651         // b_m.d 无需 rescale (其基准 max 已是全局 max),
652         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
653         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
654         md1.m = b_m.m;
655     }
656     return md1;
657 }
658
659 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
660 // Grid: (S, 1, 1)
661 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
662 // source: LeetCUDA/kernels/softmax/softmax.cu
663 template <const int NUM_THREADS = 256>
664 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
665     int tid = threadIdx.x;
666     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
667     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
668     __shared__ MD shared[NUM_WARPS];
669
670     float val = (idx < N) ? x[idx] : -FLT_MAX;
671     int warp = tid / WARP_SIZE;
672     int lane = tid % WARP_SIZE;
673
674     // 初始化: 每个线程持有一个 (max, denom) 对
675     MD md;
676     md.m = idx < N ? val : -FLT_MAX;
677     md.d = idx < N ? 1.0f : 0.0f;
678
679     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
680     md = warp_reduce_md<WARP_SIZE>(md);
681
682     if (lane == 0)
683         shared[warp] = md;
684     __syncthreads();
685
686     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
687     md = lane < NUM_WARPS ? shared[lane] : MD{-FLT_MAX, 0.0f};
688     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
689
690     // 用全局 max 和 denom 做最终 softmax
691     float d_inv = __fdividef(1.0f, md.d);
692     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径

```

```

693     if (idx < N) {
694         y[idx] = __expf(val - md.m) * d_inv;
695     }
696 }
697
698 // =====
699 // Phase 3b: RMS Normalization (1-pass reduce)
700 // =====
701 // 面试要点:
702 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
703 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
704 //   - Llama 系列使用 RMS Norm
705 //   - grid(N, K/K), block(K): 一行一个 block
706 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
707 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
708 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
709 template <const int NUM_THREADS = 128>
710 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
711     int tid = threadIdx.x;
712     int idx = blockIdx.x * blockDim.x + threadIdx.x;
713     const float epsilon = 1e-5f;
714
715     __shared__ float s_variance;
716     float value = (idx < N * K) ? x[idx] : 0.0f;
717     float variance = value * value;
718     variance = block_reduce_sum<NUM_THREADS>(variance);
719     if (tid == 0)
720         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
721     __syncthreads();
722     if (idx < N * K)
723         y[idx] = (value * s_variance) * g;
724 }
725
726 // RMS Norm + float4
727 // Grid: (N, 1, 1)
728 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
729 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
730 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
731 template <const int NUM_THREADS = 128 / 4>
732 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
733     int tid = threadIdx.x;
734     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
735     const float epsilon = 1e-5f;
736
737     __shared__ float s_variance;
738     float4 reg_x = FLOAT4(x[idx]);
739     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
740                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
741                         : 0.0f;
742     variance = block_reduce_sum<NUM_THREADS>(variance);
743     if (tid == 0)
744         s_variance = rsqrtf(variance / (float)K + epsilon);
745     __syncthreads();
746     float4 reg_y;
747     reg_y.x = reg_x.x * s_variance * g;
748     reg_y.y = reg_x.y * s_variance * g;
749     reg_y.z = reg_x.z * s_variance * g;
750     reg_y.w = reg_x.w * s_variance * g;
751     if (idx < N * K)
752         FLOAT4(y[idx]) = reg_y;
753 }
754
755 // =====

```

```

756 // Phase 3c: Layer Normalization (2-pass reduce)
757 // =====
758 // 面试要点:
759 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
760 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
761 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
762 // Grid: (N, 1, 1), 一行一个 block
763 // Block: (128, 1, 1), NUM_THREADS=K=128
764 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
765 template <const int NUM_THREADS = 128>
766 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
767     int tid = threadIdx.x;
768     int idx = blockIdx.x * blockDim.x + threadIdx.x;
769     const float epsilon = 1e-5f;
770
771     __shared__ float s_mean;
772     __shared__ float s_variance;
773     float value = (idx < N * K) ? x[idx] : 0.0f;
774
775     // Pass 1: compute mean
776     float sum = block_reduce_sum<NUM_THREADS>(value);
777     if (tid == 0)
778         s_mean = sum / (float)K;
779     __syncthreads(); // 必须等待 s_mean 对所有线程可见
780
781     // Pass 2: compute variance = (x - mean)2
782     float variance = (value - s_mean) * (value - s_mean);
783     variance = block_reduce_sum<NUM_THREADS>(variance);
784     if (tid == 0)
785         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
786     __syncthreads(); // 必须等待 s_variance 对所有线程可见
787
788     if (idx < N * K)
789         y[idx] = ((value - s_mean) * s_variance) * g + b;
790 }
791
792 // Layer Norm + float4
793 // Grid: (N, 1, 1)
794 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
795 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
796 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
797 template <const int NUM_THREADS = 128 / 4>
798 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
799     int tid = threadIdx.x;
800     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
801     const float epsilon = 1e-5f;
802
803     __shared__ float s_mean;
804     __shared__ float s_variance;
805     float4 reg_x = FLOAT4(x[idx]);
806     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
807
808     float sum = block_reduce_sum<NUM_THREADS>(value);
809     if (tid == 0)
810         s_mean = sum / (float)K;
811     __syncthreads();
812
813     float4 reg_x_hat;
814     reg_x_hat.x = reg_x.x - s_mean;
815     reg_x_hat.y = reg_x.y - s_mean;
816     reg_x_hat.z = reg_x.z - s_mean;
817     reg_x_hat.w = reg_x.w - s_mean;
818     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +

```

```

819         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
820     variance = block_reduce_sum<NUM_THREADS>(variance);
821     if (tid == 0)
822         s_variance = rsqrtf(variance / (float)K + epsilon);
823     __syncthreads();
824
825     float4 reg_y;
826     reg_y.x = reg_x_hat.x * s_variance * g + b;
827     reg_y.y = reg_x_hat.y * s_variance * g + b;
828     reg_y.z = reg_x_hat.z * s_variance * g + b;
829     reg_y.w = reg_x_hat.w * s_variance * g + b;
830     if (idx < N * K)
831         FLOAT4(y[idx]) = reg_y;
832 }
833
834 // =====
835 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
836 // =====
837 // 面试要点:
838 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
839 //       [x1']   [cos(θ)  -sin(θ)] [x1]
840 //       [x2'] = [sin(θ)   cos(θ)] [x2]
841 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
842 //   - token_pos = idx / N: token 在序列中的位置
843 //   - token_idx = idx % N: token 内的维度索引
844 //   - 输入 [seq_len, hidden_size], 输出同形状
845
846 // Grid: ((seq_len * N + 255) / 256, 1, 1)
847 // Block: (256, 1, 1)
848 // source: LeetCUDA/kernels/rope/rope.cu
849 __global__ void rope(float *x, float *out, int seq_len, int N) {
850     int idx = blockIdx.x * blockDim.x + threadIdx.x;
851     float x1 = x[idx * 2];
852     float x2 = x[idx * 2 + 1];
853     int token_pos = idx / N; // 序列位置
854     int token_idx = idx % N; // 维度索引
855
856     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
857     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
858     float sin_v = sinf(token_pos * exp_v);
859     float cos_v = cosf(token_pos * exp_v);
860
861     // 2D 旋转
862     float out1 = x1 * cos_v - x2 * sin_v;
863     float out2 = x1 * sin_v + x2 * cos_v;
864     out[idx * 2] = out1;
865     out[idx * 2 + 1] = out2;
866 }
867
868 // =====
869 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
870 // =====
871 // 面试要点 (Bank Conflict 专题):
872 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
873 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
874 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
875 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
876 //
877 // 转置的四步演进:
878 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
879 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
880 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
881 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store

```

```

882 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
883
884 // 每个线程处理 1 个元素, block(16,16)
885 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
886 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
887 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
888 // Block: (16, 16, 1)
889 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
890 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
891     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
892     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
893     if (r < row && c < col) {
894         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
895         y[c * row + r] = x[r * col + c];
896     }
897 }
898
899 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
900 // Block: (16, 16, 1)
901 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose_padded(
904     float *x, float *y, const int row, const int col) {
905     const int tx = threadIdx.x;
906     const int ty = threadIdx.y;
907
908     constexpr int TILE = 16;
909     constexpr int PAD = 1;
910     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
911     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
912
913     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
914     const int x_c = blockIdx.x * TILE + tx;
915     const int x_r = blockIdx.y * TILE + ty;
916
917     if (x_r * 4 < row && x_c < col) {
918         // Step 1: 4 rows × 1 col → smem (row-major);
919 #pragma unroll
920         for (int i = 0; i < 4; ++i) {
921             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
922         }
923         __syncthreads();
924
925         // Step 2: Transposed read from smem
926         float4 reg_trans;
927         reg_trans.x = tile[tx * 4 + 0][ty];
928         reg_trans.y = tile[tx * 4 + 1][ty];
929         reg_trans.z = tile[tx * 4 + 2][ty];
930         reg_trans.w = tile[tx * 4 + 3][ty];
931
932         // y 空间坐标: y_r = x 的列, y_c = x 的行
933         const int y_r = blockIdx.x * TILE + ty; // = x col
934         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
935
936         // Coalesced write: y[y_r][y_c .. y_c+3]
937         FLOAT4(y[y_r * row + y_c]) = reg_trans;
938     }
939 }
940
941 // =====
942 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
943 // =====
944 // 面试要点:

```

```

945 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
946 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
947 // - 不同 K 值对应不同分块策略:
948 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
949 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
950 //   K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
951 //
952 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
953
954 // ---- SGEMV K32: 基础 warp-per-row ----
955 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
956 // grid(M/4), 每个 warp 负责一行
957 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
958 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
959 // Block: (32, 4, 1), 每行 1 warp
960 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
961 // source: LeetCUDA/kernels/sgemv/sgemv.cu
962 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
963     int tx = threadIdx.x; // 0~31
964     int ty = threadIdx.y; // 0~3
965     int lane = tx % WARP_SIZE; // 0~31
966     int m = blockIdx.x * blockDim.y + ty; // 全局行号
967     if (m < M) {
968         float sum = 0.0f;
969         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
970         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
971         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
972 #pragma unroll
973         for (int w = 0; w < NUM_ITERS; ++w) {
974             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
975             int k = w * WARP_SIZE + lane;
976             sum += a[m * K + k] * x[k];
977         }
978         sum = warp_reduce_sum<WARP_SIZE>(sum);
979         // 每个 warp 处理一行, lane 0 写回结果
980         if (lane == 0)
981             y[m] = sum;
982     }
983 }
984
985 // ---- SGEMV K128: float4 向量化 ----
986 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
987 // Grid: ((M + 3) / 4, 1, 1)
988 // Block: (32, 4, 1)
989 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
990 // source: LeetCUDA/kernels/sgemv/sgemv.cu
991 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
992     int tx = threadIdx.x; // 0~31
993     int ty = threadIdx.y; // 0~3
994     int lane = tx % WARP_SIZE; // 0~31
995     int m = blockDim.y * blockIdx.x + ty;
996
997     if (m < M) {
998         float sum = 0.0f;
999         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1000         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1001 #pragma unroll
1002         for (int w = 0; w < NUM_ITERS; ++w) {
1003             int k = (w * WARP_SIZE + lane) * 4;
1004             float4 reg_x = FLOAT4(x[k]);
1005             float4 reg_a = FLOAT4(a[m * K + k]);
1006             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1007                    reg_a.w * reg_x.w);

```

```

1008     }
1009     sum = warp_reduce_sum<WARP_SIZE>(sum);
1010     if (lane == 0)
1011         y[m] = sum;
1012 }
1013 }
1014
1015 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1016 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1017 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1018 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1019 // Block: (32, 4, 1)
1020 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1021 // source: LeetCode/kernels/sgemv/sgemv.cu
1022 template <const int ROW_PER_WARP = 2>
1023 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1024     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1025     int tx = threadIdx.x;
1026     int ty = threadIdx.y;
1027     int lane = tx % WARP_SIZE;
1028     int k = lane % K_WARP_SIZE; // 0~15
1029     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1030     if (m < M) {
1031         float sum = A[m * K + k] * x[k];
1032         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1033         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1034         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1035         if (k == 0)
1036             y[m] = sum;
1037     }
1038 }
1039
1040 // =====
1041 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1042 // =====
1043 // 面试要点 (GEMM 优化五层金字塔):
1044 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1045 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1046 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1047 //   load/store 指令数 Level 4 — Tensor Core (MMA
1048 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1049 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1050 //
1051 // 计算密度递进:
1052 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1053 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1054 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1055 //
1056 // =====
1057 // Phase 7a: SGEMM (非 Tensor Core 路径)
1058 // =====
1059
1060 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1061 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1062 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1063 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1064 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1065 // Block: (32, 32, 1), 1024 线程
1066 // source: LeetCode/kernels/sgemm/sgemm.cu
1067 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1068     constexpr int BM = 32; // vec 版: 32×4 = 128
1069     constexpr int BN = 32; // vec 版: 32×4 = 128
1070     constexpr int BK = 32;

```

```

1071 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1072
1073 int bx = blockIdx.x;
1074 int by = blockIdx.y;
1075 int tx = threadIdx.x;
1076 int tid = threadIdx.y * blockDim.x + tx;
1077
1078 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1079 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1080 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1081 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1082 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1083 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1084 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1085
1086 float sum = 0.f; // 遍历完整的K, slice K;
1087 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1088     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1089     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1090     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1091     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1092     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1093     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1094     __syncthreads(); // 确保整个 smem tile 加载完毕
1095
1096 #pragma unroll
1097     for (int k = 0; k < BK; ++k) {
1098         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1099         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1100         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1101     }
1102     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1103 }
1104 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1105 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1107 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1108 }
1109
1110 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1111 // 在 Level 1 基础上引入两层优化:
1112 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1113 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1114 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1115 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1116 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1117 // 1024 x 16 = 16384 = 128 x 128 ✓
1118 //
1119 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1120 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1121 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1122 //
1123 // 加载映射 (每线程 4 个元素, float4):
1124 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1125 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1126 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1127 //
1128 // △ Bank Conflict 提示 (面试加分点):
1129 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1130 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1131 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1132 //
1133 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)

```

```

1134 // Block: (32, 32, 1), 1024 线程
1135 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1136 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1137 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1138     constexpr int BM = 128;
1139     constexpr int BN = 128;
1140     constexpr int BK = 32;
1141     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1142     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1143
1144     int bx = blockIdx.x;
1145     int by = blockIdx.y;
1146     int tx = threadIdx.x;
1147     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1148
1149     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1150     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1151     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1152     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1153     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1154     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1155
1156     int load_gmem_a_m = by * BM + load_smem_a_m;
1157     int load_gmem_b_n = bx * BN + load_smem_b_n;
1158
1159     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1160     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1161     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1162     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1163     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1164     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1165
1166     float sum[4][4] = {0.f};
1167     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1168         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1169         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1170         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1171         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1172         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1173         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1174         __syncthreads();
1175
1176     #pragma unroll
1177     for (int k = 0; k < BK; ++k) {
1178         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1179         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1180                             s_a[comp_smem_a_m_base + 1][k],
1181                             s_a[comp_smem_a_m_base + 2][k],
1182                             s_a[comp_smem_a_m_base + 3][k]};
1183         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1184                             s_b[k][comp_smem_b_n_base + 1],
1185                             s_b[k][comp_smem_b_n_base + 2],
1186                             s_b[k][comp_smem_b_n_base + 3]};
1187
1188     #pragma unroll
1189     for (int i = 0; i < 4; ++i) {
1190     #pragma unroll
1191         for (int j = 0; j < 4; ++j) {
1192             sum[i][j] += a_vals[i] * b_vals[j];
1193         }
1194     }
1195     __syncthreads();
1196 }

```

```

1197 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1198 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1199 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1200 #pragma unroll
1201 for (int i = 0; i < 4; ++i) {
1202     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1203     float4 reg_c;
1204     reg_c.x = sum[i][0];
1205     reg_c.y = sum[i][1];
1206     reg_c.z = sum[i][2];
1207     reg_c.w = sum[i][3];
1208     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1209 }
1210 }
1211 }
1212
1213 // =====
1214 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1215 // =====
1216 // 面试要点:
1217 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1218 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1219 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1220 //     寄存器)
1221 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1222 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1223 //
1224 // ★ TN 布局详解 (面试高频考点):
1225 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1226 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1227 //   N = Normal (列优先, BLAS 原生格式)
1228 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1229 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1230 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1231 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1232 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1233 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1234 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1235 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1236 //
1237 //   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1238 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1239 //   N = col-major (列优先, BLAS native = normal)
1240 //
1241 //   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1242 //   - A: row-major [M, K], B: row-major [K, N]
1243 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1244 //   - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1245 //
1246 //   TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1247 //   - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1248 //   - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1249 //   - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1250 //   MMA 指令: mma.sync.aligned.m16n8k16.row.col
1251 //   - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1252 //
1253 //   WGMMMA (Warp Group MMA, Hopper+):
1254 //   - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1255 //   - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1256 //   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1257 //     threads) 做计算
1258 //   - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1259

```

```

1260 // =====
1261 // Phase 7b-1: MMA PTX 宏定义
1262 // =====
1263
1264 // ---- gmem → smem: cp.async ----
1265 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1266 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1267 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1268 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1269 //   - wait_all: 等价于 commit_group + wait_group 0。
1270 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1271 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1272 #define CP_ASYNC_WAIT_GROUP(n) \
1273     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1274
1275 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1276 #define CP_ASYNC_CG(dst, src, bytes) \
1277     asm volatile( \
1278         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1279         "l"(src), "n"(bytes))
1280
1281 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1282 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1283 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1284 // aligned: 要求 128-bit 对齐, trans: 转置加载
1285 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1286     asm volatile( \
1287         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1288         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1289         : "r"(addr))
1290
1291 #define LDMATRIX_X2(R0, R1, addr) \
1292     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1293         : "=r"(R0), "=r"(R1) \
1294         : "r"(addr))
1295
1296 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1297 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1298 #define LDMATRIX_X2_T(R0, R1, addr) \
1299     asm volatile( \
1300         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1301         : "=r"(R0), "=r"(R1) \
1302         : "r"(addr))
1303
1304 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1305 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1306 // row.col: A 是 row-major, B 是 col-major
1307 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1308 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1309 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1310 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1311     asm volatile( \
1312         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1313         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1314         : "=r"(RD0), "=r"(RD1) \
1315         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1316         "r"(RC1))
1317
1318 // ---- MMA 辅助函数 ----
1319 // div_ceil: 整数除法向上取整
1320 #define HOST_DEVICE_INLINE __device__ __host__ inline
1321 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1322     return (a % b != 0) ? (a / b + 1) : (a / b);

```

```

1323 }
1324
1325 // =====
1326 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1327 // =====
1328 // 面试重点 — Tile Hierarchy:
1329 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1330 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1331 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1332 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1333 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1334 //
1335 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1336 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1337 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1338 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1339 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1340 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1341 //
1342 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1343 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1344 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1345 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1346 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1347 //
1348 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1349 // Block: (256, 1, 1), 8 warps
1350 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1351 // =====
1352 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1353           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1354           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1355           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1356 __global__ void __launch_bounds__(256)
1357   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1358   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1359   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1360   const int by = blockIdx.y;
1361   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1362   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1363   constexpr int BK = MMA_K; // 16
1364
1365   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1366   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1367   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1368   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1369   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1370   extern __shared__ half smem[];
1371   half *s_a = smem;
1372   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1373   constexpr int s_a_stage_offset = BM * BK; // 128*16
1374   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1375   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1376   const int warp_id = tid / WARP_SIZE; // 0~7
1377   const int lane_id = tid % WARP_SIZE; // 0~31
1378   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1379   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1380   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1381   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1382
1383   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1384   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1385   int load_smem_a_m = tid / 2; // 0~127

```

```

1386 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1387 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1388 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1389 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1390 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1391 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1392     return;
1393
1394 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1395 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1396 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1397 // MMA Tile对应到cutlass cude中的TiledMMA的概念, Value Tile对应到cutlass cude中的PermuteMNK的概念。
1398 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1399
1400 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1401 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1402 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1403
1404 // 预加载前 (K_STAGE-1) 个 stage
1405 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1406 #pragma unroll
1407 for (int k = 0; k < (K_STAGE - 1); ++k) {
1408     int load_gmem_a_k = k * BK + load_smem_a_k;
1409     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1410     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1411         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1412     );
1413     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1414
1415     int load_gmem_b_k = k * BK + load_smem_b_k;
1416     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1417     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1418     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1419         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1420     );
1421     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1422
1423     CP_ASYNC_COMMIT_GROUP();
1424 }
1425
1426 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1427 __syncthreads();
1428
1429 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1430 const int NUM_K_TILES = div_ceil(K, BK);
1431 #pragma unroll
1432 for (int k = 0; k < NUM_K_TILES; ++k) {
1433     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1434     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1435
1436     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用。因为在prefetch loop中已经
1437     // loaded (K_STAGE-1) 个 stage, 因此这里是从 tile (k+K_STAGE-1) 开始预加载
1438     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1439     // (row-major, 内维连续的是 K)
1440     if (k + K_STAGE - 1 < NUM_K_TILES) {
1441         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1442         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1443         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1444         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1445
1446         uint32_t load_smem_a_ptr =
1447             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1448                 load_smem_a_m * BK + load_smem_a_k) *

```

```

1449         sizeof(half));
1450 CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1451
1452 uint32_t load_smem_b_ptr =
1453     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1454         load_smem_b_n * BK + load_smem_b_k) *
1455         sizeof(half));
1456 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1457 CP_ASYNC.COMMIT_GROUP();
1458 }
1459
1460 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1461 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1462 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1463 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1464 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1465
1466 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1467 #pragma unroll
1468 for (int i = 0; i < VAL_TILE_M; ++i) {
1469     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1470     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1471     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1472     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1473     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1474     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1475     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1476     uint32_t lane_smem_a_ptr =
1477         (smem_a_base_ptr +
1478         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1479         sizeof(half));
1480     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1481 }
1482
1483 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1484 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1485 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1486 #pragma unroll
1487 for (int j = 0; j < VAL_TILE_N; ++j) {
1488     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1489     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1490     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1491     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1492     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1493     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1494     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1495     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1496     uint32_t lane_smem_b_ptr =
1497         (smem_b_base_ptr +
1498         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1499         sizeof(half));
1500     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1501 }
1502
1503 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1504 #pragma unroll
1505 for (int i = 0; i < VAL_TILE_M; ++i) {
1506     #pragma unroll
1507     for (int j = 0; j < VAL_TILE_N; ++j) {
1508         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1509             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1510             RB[j][0], RB[j][1], // B fragment
1511             RC[i][j][0], RC[i][j][1]);

```

```

1512 }
1513 }
1514
1515 // 自适应等待：流水线满载期用 K_STAGE-2，尾部排空用 0
1516 if (k + K_STAGE - 1 < NUM_K_TILES) {
1517     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1518 } else { // 对于尾部的 k，等待所有剩余的 cp.async 完成
1519     CP_ASYNC_WAIT_GROUP(0);
1520 }
1521 __syncthreads();
1522 }
1523
1524 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1525 {
1526     for (int i = 0; i < VAL_TILE_M; ++i) {
1527         // RC[VAL_TILE_M][VAL_TILE_N][2] 中 RC[...][...][2] 中保存了2个 uint32
1528         // 寄存器，每个uint32 寄存器表示两个临近的fp16值：{c0,c1}，然后RC[...][...][0]
1529         // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上（不同物理行，跨8x8）
1530         // 同一个位置上的元素，也就是，实际代表了2个不同的行的元素，因此要分开RC0，RC1；
1531         // RC0表示第一行，8个half，可以用4个uint32寄存器来装；同理，RC1表示第二行。
1532         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1533         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1534         // =====
1535         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1536         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1537         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1538         //
1539         //      cols: c0-1   c2-3   c4-5   c6-7
1540         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1541         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1542         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1543         //  ...
1544         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1545         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1546         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1547         // r10:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1548         //  ...
1549         // r15:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1550         //
1551         // Within a 4-lane group (e.g. T0-T3 for row 0):
1552         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1553         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1554         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0，4 次 shuffle 凑齐
1555         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}，再由 lane%4==0 128-bit store.
1556         // =====
1557 #pragma unroll
1558         for (int j = 0; j < VAL_TILE_N; ++j) {
1559             RC0[j][0] = RC[i][j][0];
1560             RC1[j][0] = RC[i][j][1];
1561             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1562             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1563             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1564             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1565             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1566             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1567         }
1568         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1569         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行)：
1570         //
1571         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1572         // |-----|-----|-----|-----|
1573         // | 0~3   | 0       | row 0     | row 8     |
1574         // | 4~7   | 1       | row 1     | row 9     |

```

```

1575 // | 8~11 | 2 | row 2 | row 10 |
1576 // | 12~15 | 3 | row 3 | row 11 |
1577 // | 16~19 | 4 | row 4 | row 12 |
1578 // | 20~23 | 5 | row 5 | row 13 |
1579 // | 24~27 | 6 | row 6 | row 14 |
1580 // | 28~31 | 7 | row 7 | row 15 |
1581 // |-----|
1582 if (lane_id % 4 == 0) {
1583     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1584     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1585     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1586 #pragma unroll
1587     for (int j = 0; j < VAL_TILE_N; ++j) {
1588         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1589         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1590         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1591         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1592         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1593         // 128-bit store: 一次写入 8 个 half
1594         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1595             *reinterpret_cast<float4*>(&RC0[j][0]);
1596         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1597             *reinterpret_cast<float4*>(&RC1[j][0]);
1598     }
1599 }
1600 }
1601 }
1602 }
1603
1604 // =====
1605 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1606 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1607 // =====
1608 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1609 // - 问题: ldmatrix 以 8×8 片段 (m8n8) 从 smem 加载 16×16 的矩阵。同一 warp 中
1610 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1611 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1612 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1613 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1614 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1615 //   rows 8~11: repeat rows 0~3 pattern
1616 //   rows 12~15: repeat rows 4~7 pattern
1617 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1618 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1619 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1620 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1621 //
1622 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1623 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1624
1625 // ---- Swizzle 辅助函数 ----
1626
1627 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1628 // i: row index; j: col index.
1629 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1630 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1631 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1632 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1633 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1634 template <const int kColStride = 16, const int kStep = 8>
1635 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1636     // for col_stride > 16, we have to permute it using col major ZigZag order.
1637     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].

```

```

1638 static_assert(kColStride <= 16, "kColStride must <= 16");
1639 // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1640 static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1641 static_assert(kColStride % kStep == 0,
1642             "kColStride must be multiple of kStep.");
1643 if constexpr (kStep == 8) {
1644     return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1645 } else {
1646     static_assert(kStep == 4);
1647     return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1648 }
1649 }
1650
1651 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1652 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1653 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1654 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1655 // TMA中的SWIZZLE_32B pattern.
1656 // -----
1657 // -col 0~16, step 8--
1658 // -----
1659 // | row 0 | (0, 8) |
1660 // | row 1 | (0, 8) |
1661 // | row 2 | (0, 8) |
1662 // | row 3 | (0, 8) |
1663 // -----
1664 // | row 4 | (8, 0) |
1665 // | row 5 | (8, 0) |
1666 // | row 6 | (8, 0) |
1667 // | row 7 | (8, 0) |
1668 // -----
1669 // | row 8 | (0, 8) |
1670 // | row 9 | (0, 8) |
1671 // | row 10 | (0, 8) |
1672 // | row 11 | (0, 8) |
1673 // -----
1674 // | row 12 | (8, 0) |
1675 // | row 13 | (8, 0) |
1676 // | row 14 | (8, 0) |
1677 // | row 15 | (8, 0) |
1678 // -----
1679 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1680 template <const int kMmaAtomK = 16>
1681 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1682     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1683 }
1684
1685 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern)。
1686 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1687 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8)。
1688 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1689 template <const int kMmaAtomK = 16>
1690 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1691     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1692 }
1693
1694 // =====
1695 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1696 // =====
1697 // 与 Phase 7b-2 (hgemm_mma_stages_tn) 的核心区别:
1698 // - 所有 smem 地址计算加 swizzle_permuted_A_j/swizzle_permuted_B_j, 消除 bank conflict
1699 // - VAL_TILE_M/N → VAL_TILE_M/N (与源 kernel 命名一致)
1700 // - 其余 tile hierarchy、pipeline、epilogue 与统一循环版完全一致

```

```

1701 //
1702 // Tile Hierarchy:
1703 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1704 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1705 //   WARP Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1706 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1707 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1708 //
1709 // TN 布局: C[MxN] = A[MxK] × BT[NxK]
1710 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1711 //   - BT[N][K]: row-major → 全局索引 B[n*K+k]
1712 //   - ldmatrix A: x4 (非转置), A row-major 原生匹配
1713 //   - ldmatrix B: x2 (非转置), BT row-major 逐行加载 = B 的列
1714 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1715 //
1716 // 统一循环设计 (同 hgemmm_mma_stages_tn) :
1717 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1718 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1719 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1720 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1721 //
1722 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1723 // Block: (256, 1, 1), 8 warps
1724 // source: LeetCUDA/kernels/hgemmm/mma/swizzle/hgemmm_mma_stage_tn_swizzle.cu
1725 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1726           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1727           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1728           const int K_STAGE = 3,
1729           const bool BLOCK_SWIZZLE = false>
1730 __global__ void __launch_bounds__(256)
1731   hgemmm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1732   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1733   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1734   const int by = blockIdx.y;
1735   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1736   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1737   constexpr int BK = MMA_K; // 16
1738
1739   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B (与 hgemmm_mma_stages_tn 相同)
1740   extern __shared__ half smem[];
1741   half *s_a = smem;
1742   half *s_b = smem + K_STAGE * BM * BK;
1743   constexpr int s_a_stage_offset = BM * BK; // 128*16
1744   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BT row-major
1745   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1746   const int warp_id = tid / WARP_SIZE; // 0~7
1747   const int lane_id = tid % WARP_SIZE; // 0~31
1748   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1749   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1750
1751   // 线程到 global memory 的映射 (用于加载 A 和 B)
1752   int load_smem_a_m = tid / 2; // 0~127
1753   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1754   int load_smem_b_n = tid / 2; // 0~127 → BT 的 N 方向 (row-major 的行)
1755   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → BT 的 K 方向 (row-major 的列)
1756   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号
1757   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号
1758   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1759     return;
1760
1761   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1762   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1763   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。

```

```

1764 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1765 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1766
1767 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1768 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1769 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1770
1771 // 预加载前 (K_STAGE-1) 个 stage
1772 // * swizzle 区别: cp.async 的目标 smem 地址加 swizzle_permuted_A_j/swizzle_permuted_B_j
1773 #pragma unroll
1774 for (int k = 0; k < (K_STAGE - 1); ++k) {
1775     int load_gmem_a_k = k * BK + load_smem_a_k;
1776     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1777     int load_gmem_b_k = k * BK + load_smem_b_k;
1778     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1779
1780     uint32_t load_smem_a_ptr =
1781         (smem_a_base_ptr +
1782          (k * s_a_stage_offset + load_smem_a_m * BK +
1783           swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1784           sizeof(half));
1785     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1786
1787     uint32_t load_smem_b_ptr =
1788         (smem_b_base_ptr +
1789          (k * s_b_stage_offset + load_smem_b_n * BK +
1790           swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1791           sizeof(half));
1792     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1793
1794     CP_ASYNC.COMMIT_GROUP();
1795 }
1796
1797 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1798 __syncthreads();
1799
1800 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1801 const int NUM_K_TILES = div_ceil(K, BK);
1802 #pragma unroll
1803 for (int k = 0; k < NUM_K_TILES; ++k) {
1804     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1805     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1806
1807     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1808     // swizzle 区别: smem 地址加 swizzle_permuted_*_j
1809     if (k + K_STAGE - 1 < NUM_K_TILES) {
1810         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1811         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1812         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1813         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1814
1815         uint32_t load_smem_a_ptr =
1816             (smem_a_base_ptr +
1817              (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1818               swizzle_permuted_A_j<MMA_K>(load_smem_a_m, load_smem_a_k)) *
1819               sizeof(half));
1820         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1821
1822         uint32_t load_smem_b_ptr =
1823             (smem_b_base_ptr +
1824              (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1825               swizzle_permuted_B_j<MMA_K>(load_smem_b_n, load_smem_b_k)) *
1826               sizeof(half));

```

```

1827     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1828     CP_ASYNC.COMMIT_GROUP();
1829 }
1830
1831 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1832 // * swizzle 区别: ldmatrix 的源 smem 地址也加 swizzle_permuted_*_j
1833 uint32_t RA[VAL_TILE_M][4];
1834 uint32_t RB[VAL_TILE_N][2];
1835
1836 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1837 #pragma unroll
1838 for (int i = 0; i < VAL_TILE_M; ++i) {
1839     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1840     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1841     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1842     uint32_t lane_smem_a_ptr =
1843         (smem_a_base_ptr +
1844          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1845           swizzle_permuted_A_j < MMA_K > (lane_smem_a_m, lane_smem_a_k)) *
1846           sizeof(half));
1847     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1848 }
1849
1850 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1851 #pragma unroll
1852 for (int j = 0; j < VAL_TILE_N; ++j) {
1853     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1854     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1855     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1856     uint32_t lane_smem_b_ptr =
1857         (smem_b_base_ptr +
1858          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1859           swizzle_permuted_B_j < MMA_K > (lane_smem_b_n, lane_smem_b_k)) *
1860           sizeof(half));
1861     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1862 }
1863
1864 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1865 #pragma unroll
1866 for (int i = 0; i < VAL_TILE_M; ++i) {
1867     #pragma unroll
1868     for (int j = 0; j < VAL_TILE_N; ++j) {
1869         HMMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1870                   RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1871                   RB[j][0], RB[j][1], // B fragment
1872                   RC[i][j][0], RC[i][j][1]);
1873     }
1874 }
1875
1876 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1877 if (k + K_STAGE - 1 < NUM_K_TILES) {
1878     CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1879 } else {
1880     CP_ASYNC.WAIT_GROUP(0);
1881 }
1882 __syncthreads();
1883 }
1884
1885 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1886 // 与 hgemm_mma_stages_tn 完全一致的 epilogue 模式
1887 {
1888     for (int i = 0; i < VAL_TILE_M; ++i) {
1889         uint32_t RC0[VAL_TILE_N][4];

```

```

1890     uint32_t RC1[VAL_TILE_N][4];
1891 #pragma unroll
1892     for (int j = 0; j < VAL_TILE_N; ++j) {
1893         RC0[j][0] = RC[i][j][0];
1894         RC1[j][0] = RC[i][j][1];
1895         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1896         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1897         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1898         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1899         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1900         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1901     }
1902     if (lane_id % 4 == 0) {
1903         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1904         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1905 #pragma unroll
1906         for (int j = 0; j < VAL_TILE_N; ++j) {
1907             int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1908             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1909             int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1910             int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1911             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1912                 *reinterpret_cast<float4*>(&RC0[j][0]);
1913             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1914                 *reinterpret_cast<float4*>(&RC1[j][0]);
1915         }
1916     }
1917 }
1918 }
1919 }
1920
1921 // =====
1922 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
1923 // =====
1924 // 面试要点 (CuTe vs 手写 MMA PTX) :
1925 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
1926 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
1927 //   - 与手写 MMA PTX (Phase 7b) 的对比:
1928 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
1929 //     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
1930 //       cute::copy / cute::gemm 自动展开为高效指令序列
1931 //   - 核心抽象 ("CuTe 五要素") :
1932 //     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
1933 //     2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
1934 //     3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
1935 //     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
1936 //     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
1937 //
1938 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
1939 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
1940 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
1941 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
1942 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
1943 //
1944 // 调参口诀 (面试速记) :
1945 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
1946 //   Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
1947 //   重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
1948 //     1. 连续的 2^M 个元素组成二维空间中的一个基本元素;
1949 //     2. 连续的 2^S 个基本元素组成一行;
1950 //     3. 二维空间包含 2^B 行;
1951 //     4. 对二维坐标做列置换: icol' = irow ^ icol;
1952 //     5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。

```

```

1953 // 因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
1954 // CuTe 的位级实现等价于:
1955 //     offset' = offset ^ ((offset & YYY) >> S),
1956 //     YYY = ((1 << B) - 1) << (M + S)。
1957 // 对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
1958 // 二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
1959 // 一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
1960 // kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
1961 // EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
1962 // 与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
1963 //
1964 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
1965 // - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
1966 // - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
1967 // - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
1968 //
1969 // ★ 本实现的 Tile 配置:
1970 // - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
1971 // - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
1972 // - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
1973 //   BN=256 不是由“8x2x16”直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
1974 // - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
1975 //   的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
1976 // - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
1977 // - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
1978 //
1979 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
1980 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
1981 // =====
1982
1983 #if defined(NOTES_V2_ENABLE_CUTE)
1984
1985 #include <cute/tensor.hpp>
1986
1987 // =====
1988 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
1989 // =====
1990 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
1991 // - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [KxN])
1992 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
1993 //
1994 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
1995 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
1996 // 2. 主循环 over K tiles:
1997 //   a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
1998 //   b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
1999 //   c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2000 //   d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2001 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2002 //
2003 // 面试对比 — 手写 MMA vs CuTe 代码量:
2004 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2005 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2006 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
2007           typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
2008           typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
2009           typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
2010           typename S2GCopyC>
2011 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2012                                           int n, int k) {
2013     using namespace cute;
2014
2015     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。

```

```

2016 extern __shared__ T shm_data[];
2017 T *Ashm = shm_data;
2018 T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2019
2020 int idx = threadIdx.x;
2021 int ix = blockIdx.x; // N 方向 block 索引 (本实现默认不开启 BlockSwizzle)
2022 int iy = blockIdx.y; // M 方向 block 索引
2023 // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2024 // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2025 if (iy * BM >= m || ix * BN >= n) return;
2026
2027 // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2028 // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2029 Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2030                        make_stride(k, Int<1>{}));
2031 Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2032                        make_stride(k, Int<1>{}));
2033 Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2034                        make_stride(n, Int<1>{}));
2035
2036 // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2037 // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2038 //   gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2039 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2040 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2041 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2042
2043 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2044 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2045 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2046
2047 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2048 TiledMMA tiled_mma;
2049 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2050 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2051 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2052 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2053 clear(tCrD); // 累加器清零
2054
2055 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2056 G2SCopyA g2s_tiled_copy_a;
2057 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2058 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2059 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2060
2061 G2SCopyB g2s_tiled_copy_b;
2062 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2063 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2064 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2065
2066 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2067 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2068 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2069 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2070 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2071 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2072
2073 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2074 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2075 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2076 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2077
2078 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。

```

```

2079     int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2080 #pragma unroll
2081     for (int istage = 0; istage < kStage - 1; ++istage) {
2082         cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2083                   tAsA_copy(_, _, _, istage));
2084         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2085                   tBsB_copy(_, _, _, istage));
2086         cp_async_fence();
2087         ++itle_to_read;
2088         ++ismem_write;
2089     }
2090     cp_async_wait<kStage - 2>();
2091     __syncthreads();
2092
2093     // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2094     // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2095     // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2096     int num_k_tiles = k / BK;
2097     // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2098     cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2099     cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2100
2101 #pragma unroll 1
2102     for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2103         // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2104         //
2105         // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2106         //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2107         //   - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据(ldmatrix),
2108         //   所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2109         //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2110         //   同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2111         //   在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2112         //   无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2113         //       for (i=0; i<VAL_TILE_M; ++i)
2114         //           for (j=0; j<VAL_TILE_N; ++j)
2115         //               MMA16816(RC[i][j][0], RC[i][j][1], ...);
2116         //
2117         // CUTLASS 推导链 (mma_atom.hpp) :
2118         //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2119         //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2120         //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2121         //   做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2122         //       K 被分为 BK/kMmaPK 个 perm-K slice
2123         //       随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2124         //       每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2125         //       → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2126         //   本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2127         int num_k_steps = size<2>(tCrA);
2128
2129 #pragma unroll
2130         for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2131             int k_step_next = (k_step + 1) % num_k_steps;
2132
2133             if (k_step == num_k_steps - 1) {
2134                 // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2135                 cp_async_wait<kStage - 2>();
2136                 __syncthreads();
2137                 ismem_read = (ismem_read + 1) % kStage;
2138             }
2139
2140             // S→R: 加载下一组 A/B fragment 到寄存器。
2141             //

```

```

2142 // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2143 //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2144 // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**::
2145 //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2146 //   让 S→R 延迟与 MMA 计算重叠。
2147 // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2148 cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2149           tCrA_view(_, _, k_step_next));
2150 cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2151           tCrB_view(_, _, k_step_next));
2152
2153 if (k_step == 0) {
2154     // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2155     if (itile_to_read < num_k_tiles) {
2156         cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2157                 tAsA_copy(_, _, _, ismem_write));
2158         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2159                 tBsB_copy(_, _, _, ismem_write));
2160         ++itile_to_read;
2161         ismem_write = (ismem_write + 1) % kStage;
2162     }
2163     cp_async_fence();
2164 }
2165
2166 // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2167 // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2168 cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2169 }
2170 }
2171
2172 // Epilogue: D 寄存器 → shared memory → global memory。
2173 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2174 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2175 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2176 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2177 //   解释同一段地址。当前 wrapper 的固定配置下:
2178 //   一个 A stage: 128 x 32 = 4096 个 half;
2179 //   C scratchpad: 32 x 32 x 4 = 4096 个 half。
2180 // 两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2181 // 是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2182 // C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2183 // 检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2184 //
2185 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2186 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2187 // 用它写 C, S2G 也用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2188 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2189
2190 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2191 // R2SCopyAtomC = CopyAtom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2192 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2193 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2194 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2195 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2196 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2197 //
2198 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2199 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2200 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2201 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2202 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2203 // 可将它与主循环的 s2r_thr_copy_a.retire_D(tCrA) 对照: 二者都是为了让已有
2204 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination

```

```

2205 // 角色，而不是另一次 memory transfer。
2206 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2207 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2208 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2209 // R2S TiledCopy 的 destination mapping 切分 sC。推导链：
2210 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2211 //   -> TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2212 //   -> make_tiled_copy_C 的 tiler = (32, 32)
2213 //   -> SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2214 //   -> partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2215 //   -> 结果 shape 的 M/N remainder = _1, _1
2216 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch，而不是
2217 // “thread-level tensor 自动只剩 CPY” 这一条规则。_1 表示该逻辑 mode 的 extent
2218 // 是 1，并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2219 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2220
2221 // S2G TiledCopy: shared memory -> global memory (128-bit store)
2222 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2223 S2GCopyC s2g_tiled_copy_c;
2224 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2225 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生，因而保留相同的 pipe=4；前者是
2226 // S2G source mapping，后者是 R2S destination mapping。tCgC_s2g 则面向完整
2227 // gD=(BM,BN)=(128,256)，相对于 S2G copy tiler 仍有非平凡的 M/N repetition，
2228 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2229 // 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2230 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2231 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2232
2233 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode：
2234 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2235 //   -> Tensor(CPY, (CPY_M, CPY_N))
2236 // 它只重组 Tensor 的 layout，不分配内存、不移动 data()，也不改变元素访问的物理地址。
2237 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
2238 // i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说“合并 mode”，
2239 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2240 // 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
2241 // mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(…) 可作为一个标量
2242 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2243 // 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
2244 // pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2245 //
2246 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group？
2247 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N)；
2248 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐；
2249 //   - 两者使用相同的 [1,3) 合并后，源/目的 Tensor 仍保持相同的坐标空间，
2250 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2251 //
2252 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
2253 // shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
2254 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2255 // 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
2256 // 再从同一个 slot 写回 global memory。
2257 // 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
2258 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
2259 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2260 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2261 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2262
2263 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2264 // 双层循环的语义 (注意 step 与 CPY_N 无关)：
2265 //   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2266 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
2267 //   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入

```

```

2268 // 第 j 号 C scratchpad pipe slot, 再读出写回 global。
2269 //
2270 // 这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2271 // TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2272 // 通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2273 // CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2274 // 唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2275 //
2276 // 第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2277 // 各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2278 // 写入和读取命中同一段 shared memory。
2279 #pragma unroll
2280 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2281     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2282     #pragma unroll
2283     for (int j = 0; j < step; ++j) {
2284         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2285         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2286         // 成输出元素类型 T 的临时 payload。
2287         //
2288         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2289         // dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2290         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2291         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2292         //
2293         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2294         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2295         // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2296         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2297         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2298         // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2299         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2300         // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2301         //
2302         // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2303         // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2304         // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2305         // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2306         // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2307         // 保持同源实现的通用 staging 写法。
2308         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2309         cute::copy(tCrC_r2sx(_, i + j), t);
2310         // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2311         // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2312         //
2313         // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2314         // r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2315         // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2316         // → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
2317         // → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2318         // 走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2319         // → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2320         //
2321         // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2322         // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2323         // - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2324         //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2325         // - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2326         //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2327         //   计算了当前线程每个 val 在 smem 中的物理 offset
2328         // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2329         // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2330         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));

```

```

2331 }
2332 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2333 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2334 __syncthreads();
2335
2336 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2337 #pragma unroll
2338 for (int j = 0; j < step; ++j) {
2339     // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2340     // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2341     // (如 st.global.v4.f32 或 st.global.b128)。
2342     //
2343     // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2344     //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2345     //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2346     //   → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1
2347     //   → 直接 copy_atom.call(src, dst)
2348     //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2349     //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2350     //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2351     //
2352     // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2353     //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2354     //   用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2355     //   在 sC (shared memory) 中的物理 offset。
2356     //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2357     //   = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2358     //   因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2359     //   M/N remainder。
2360     //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2361     //   → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2362     //   因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2363     //   repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2364     //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2365     //   因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2366     cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2367 }
2368 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2369 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2370 __syncthreads();
2371 }
2372
2373 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2374 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2375 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2376 // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
2377 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2378 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2379 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2380 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2381 }
2382
2383 // =====
2384 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2385 // =====
2386 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2387 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2388 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2389 //
2390 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2391 //
2392 // 面试常问: 「CuTe 为什么比手写 PTX 简洁? 」
2393 // 答: 手写需要:

```

```

2394 // 1) 手动计算每线程的 smem 地址偏移
2395 // 2) 手动计算 ldmatrix 的 lane 映射
2396 // 3) 手动插入 swizzle XOR 到每个地址计算点
2397 // 4) 手动做 epilogue 的 warp shuffle 编排
2398 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2399 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2400 // =====
2401 template <typename T, const int Stages = 2>
2402 void launch_hgemmm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2403     using namespace cute;
2404
2405     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2406     //
2407     // kernel 用法对照:
2408     //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2409     //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2410     //   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2411     //             local_tile(D, ...) → gD 的列方向
2412     //   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2413     //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2414     //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2415     auto BM = Int<128>{};
2416     auto BN = Int<256>{};
2417     auto BK = Int<32>{};
2418     auto KStage = Int<Stages>{};
2419     auto kSmemLayoutCBatch = Int<4>{};
2420
2421     // — SmemLayoutA / SmemLayoutB —
2422     // kernel 用法:
2423     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2424     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2425     // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2426     //
2427     // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2428     //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2429     //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2430     //   - B=3: 共有 2^3=8 行。
2431     // 于是 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2432     // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2433     // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2434     // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2435     // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2436     // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2437     // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2438     // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2439     // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2440     using SmemLayoutAtom = decltype(composition(
2441         Swizzle<3, 3, 3>{},
2442         make_layout(make_shape(Int<8>{}, Int<BK>{}),
2443             make_stride(Int<BK>{}, Int<1>{}))));
2444     using SmemLayoutA = decltype(tile_to_shape(
2445         SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2446     using SmemLayoutB = decltype(tile_to_shape(
2447         SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2448
2449     // — MMA (TiledMMA) —
2450     // kernel 用法:
2451     // TiledMMA tiled_mma;
2452     // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2453     // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2454     // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2455     // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2456     // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);

```

```

2457 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2458 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2459 //
2460 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2461 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2462 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2463 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2464 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2465 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2466 using mma_traits = MMA_Traits<mma_op>;
2467 using mma_atom = MMA_Atom<mma_traits>;
2468 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2469
2470 static constexpr int kMmaEURepeatM = 2;
2471 static constexpr int kMmaEURepeatN = 2;
2472 static constexpr int kMmaEURepeatK = 1;
2473 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2474 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2475 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2476 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2477
2478 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2479     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2480 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2481 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2482
2483 // — G2SCopyA / G2SCopyB —
2484 // kernel 用法:
2485 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2486 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2487 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2488 // G→S: 128-bit cp.async。ThrLayout{32,4} × ValLayout{1,8}:
2489 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2490 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2491 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2492 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2493 using G2SCopyA = decltype(make_tiled_copy(
2494     g2s_copy_atom{},
2495     make_layout(make_shape(Int<32>{}, Int<4>{}),
2496         make_stride(Int<4>{}, Int<1>{}))),
2497     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2498 using G2SCopyB = G2SCopyA;
2499
2500 // — S2RCopyAtomA / S2RCopyAtomB —
2501 // kernel 用法:
2502 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2503 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2504 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2505 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2506 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2507 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout。
2508 using s2r_copy_op = SM75_U32x4_LDSM_N;
2509 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2510 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2511 using S2RCopyAtomA = s2r_copy_atom;
2512 using S2RCopyAtomB = s2r_copy_atom;
2513
2514 // — SmemLayoutC —
2515 // kernel 用法:
2516 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2517 // 复用当前 A stage 的空间作为 C scratchpad
2518 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2519 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4

```

```

2520 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32x32 (MMA tile 大小),
2521 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2522 // 的 kStage K-tile pipeline。
2523 using SmemLayoutAtomC = decltype(composition(
2524     Swizzle<3, 3, 3>{},
2525     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2526         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2527 using SmemLayoutC = decltype(tile_to_shape(
2528     SmemLayoutAtomC{},
2529     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2530
2531 // — R2SCopyAtomC —
2532 // kernel 用法:
2533 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2534 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2535 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2536 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2537 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2538 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2539
2540 // — S2GCopyC —
2541 // kernel 用法:
2542 // S2GCopyC s2g_tiled_copy_c;
2543 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2544 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared-global wide store。
2545 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2546 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2547 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2548 using S2GCopyC = decltype(make_tiled_copy(
2549     S2GCopyAtomC{},
2550     make_layout(make_shape(Int<32>{}, Int<4>{}),
2551         make_stride(Int<4>{}, Int<1>{})),
2552     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2553
2554 // — Grid/Block 配置 —
2555 // kernel 用法:
2556 // int idx = threadIdx.x;          // 0..127
2557 // int ix = blockIdx.x;            // N 方向 CTA 坐标
2558 // int iy = blockIdx.y;            // M 方向 CTA 坐标
2559 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2560 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2561 // M % BM == 0、N % BN == 0、K % BK == 0。
2562 int BX = (N + BN - 1) / BN;
2563 int BY = (M + BM - 1) / BM;
2564 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2565 dim3 grid(BX, BY);
2566
2567 // — Dynamic Shared Memory 大小 —
2568 // kernel 用法:
2569 // extern __shared__ T shm_data[];
2570 // T *Ashm = shm_data;
2571 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2572 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2573 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2574 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2575 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2576 // 能放进一个 A stage (128*32 = 4096)。
2577 static constexpr int shm_size_AB =
2578     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2579 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2580 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2581     "C shared memory must fit within one A pipe");
2582 static constexpr int kShmSize =

```

```

2583     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2584
2585     cudaFuncSetAttribute(
2586         hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2587             SmemLayoutA, SmemLayoutB, SmemLayoutC,
2588             S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2589             S2GCopyAtomC, S2GCopyC>,
2590         cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2591
2592     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2593         SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2594         S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC>
2595     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2596 }
2597
2598 #endif /* NOTES_V2_ENABLE_CUTE */
2599
2600 // =====
2601 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2602 // =====
2603 // 面试要点 (WGMMMA vs MMA 对比) :
2604 //   - MMA: warp 级 (32 threads) , 同步执行
2605 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
2606 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2607 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2608 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2609 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2610
2611 // ---- WGMMMA 辅助函数 ----
2612 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2613 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2614 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
2615 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
2616 //     N=0 → 等所有 group 完成。* 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
2617 //     都是 "wait until only N or fewer groups are pending"。
2618 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2619 #define WGMMMA_COMMIT_GROUP() \
2620     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2621 #define WGMMMA_WAIT_GROUP(n) \
2622     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2623
2624 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2625 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2626 //   encoded = (x & 0x3FFFF) >> 4
2627 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2628 // >>4 是因为为 shared memory 地址/偏移必须 16B 对齐 (2^4) , 低 4 bit 恒为 0,
2629 // 可省略以扩大可表示范围。
2630 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
2631 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4)
2632
2633 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2634 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2635 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2636 //
2637 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2638 // -----
2639 //   | 位域           | 范围       | 大小 | 含义
2640 //   |-----|-----|-----|-----|
2641 //   | start_address   | [0, 14)    | 14   | smem 基址编码
2642 //   | (unused)        | [14, 16)   | 2    | -
2643 //   | leading_byte_offset | [16, 30)  | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2644 //   | (unused)        | [30, 32)   | 2    | -
2645 //   | stride_byte_offset | [32, 46)   | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距

```

```

2646 // | (unused) | [46, 49) | 3 | -
2647 // | base_offset | [49, 52) | 3 | swizzle pattern 偏移
2648 // | (unused) | [52, 62) | 10 | -
2649 // | layout_type | [62, 64) | 2 | swizzle 模式
2650 // -----
2651 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2652 //
2653 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2654 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2655 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2656 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2657 // - 这是 stride_byte_offset=1024 的来源
2658 //
2659 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2660 // 此处填 16 仅为占位, 编码后为 1。
2661 //
2662 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2663 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2664 //
2665 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2666 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2667 __device__ inline uint64_t make_smem_desc(half *ptr) {
2668 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2669 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2670 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2671
2672 // 从全零开始: 所有位域初始化为 0。
2673 uint64_t desc = 0x0000000000000000;
2674
2675 // — bits [0, 14): start_address —
2676 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2677 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2678 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2679 desc |= SMEM_DESC_ENCODE(addr);
2680
2681 // — bits [16, 30): leading_byte_offset —
2682 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2683 // << 16 将编码值放到 bits [16, 30)。
2684 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2685 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2686 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2687 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2688 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2689 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2690
2691 // — bits [32, 46): stride_byte_offset —
2692 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2693 // << 32 将编码值放到 bits [32, 46)。
2694 // K-Major 下定义为“从首 8 行到次 8 行的字节偏移” (PTX ISA §9.7.15.5.1.2.1.2) 。
2695 // 1024 的来源 (K-Major + 128B swizzle + half) :
2696 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2697 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2698 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2699 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2700
2701 // — bits [62, 64): layout_type (swizzle mode) —
2702 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2703 // 1 = 128B swizzle mode。
2704 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2705 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2706 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2707 desc |= 1llu << 62;
2708

```

```

2709     return desc;
2710 }
2711
2712 // ---- WGMMA PTX 指令宏 ----
2713 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2714 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2715 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2716 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2717 // 每线程 64 half = 32 uint32。
2718 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2719 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2720 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2721 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2722 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2723 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2724                                     TransB) \
2725 { \
2726     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2727     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2728     asm volatile( \
2729         "{\n" \
2730         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2731         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
2732         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
2733         " %16, %17, %18, %19, %20, %21, %22, %23, " \
2734         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
2735         " %32, " \
2736         " %33, " \
2737         " %34, %35, %36, %37, %38;\n" \
2738         "}\n" \
2739         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2740         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2741         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2742         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2743         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2744         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2745         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2746         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2747         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
2748         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2749         "n"(int32_t(TransB))); \
2750 }
2751
2752 // ---- TMA Shared Memory Layout ----
2753 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储  $A[BM \times BK] + B[BK \times BN]$ 
2754 //
2755 // 每个 stage 包含两块 smem:
2756 //   A tile:  $[BM, BK]$  row-major  $\rightarrow BK \times BM$  个 half, 地址连续
2757 //   B tile: TMA 按  $[BN, BK]$  物理写入 (详见下方 ★),  $BN \times BK$  个 half
2758 //
2759 // ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
2760 //
2761 //   上层数据: 源矩阵  $B^T [N, K]$  row-major (TN 布局, B 的列以行优先形式存储)。
2762 //   物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为  $[BN, BK]$  row-major
2763 //             ( $BN=128$  行,  $BK=64$  列, 每行 64 个连续的 half)。
2764 //   TMA 的 smem_box_shape = ( $BK=64$ ,  $BN=128$ ) 定义了每个 tile 的 shape,
2765 //   TMA 按行写入, 行内  $BK$  个 half 连续  $\rightarrow$  物理上就是  $[BN][BK]$ 。
2766 //   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2767 //             通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按  $[BK, BN]$  寻址。
2768 //             实际  $[BN, BK] \rightarrow$  逻辑  $[BK, BN]$  的"转置"是通过 descB 中的 128B
2769 //             swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2770 //
2771 //   stride_byte_offset=1024 的来源 ( $K\text{-major} + 128B \text{ swizzle} + f16$ , PTX ISA §9.7.15.5.1.2.1.2) :

```

```

2772 //      128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2773 //      stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2774 //                          = 8 rows × (BK=64) halves × 2 bytes = 1024。
2775 //      这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2776 //
2777 // 与 MMA(TN) 的对比:
2778 //      MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2779 //      WGMMMA:    smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2780 //      简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2781 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2782     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2783     // B tile 标记:
2784     // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2785     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2786     //   以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2787     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2788     //   但写 BN×BK 更能体现物理存储形态
2789     alignas(128) half B[BN * BK * QSIZE];
2790 };
2791
2792 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2793 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
2794 //
2795 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2796 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2797 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2798 //
2799 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2800 //   将 A/B tile 从 HBM 搬到 shared memory。
2801 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2802 //
2803 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2804 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2805 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2806 //
2807 // 本节重点理解:
2808 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2809 //   即可搬运整个 2D tile, 无需所有线程参与。
2810 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2811 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2812 //   Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2813 //
2814 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
2815 // WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2816 // K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
2817 // M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
2818 // N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
2819 // Block Tile:       C[128,128] = A[128,64] × B[64,128]
2820 // Threads:          256 = 2 warpgroups × 128 threads/warpgroup
2821 //   Producer(WG0):  128 threads (仅 thread 0 做 TMA 提交)
2822 //   Consumer(WG1):  128 threads = 4 warps (全部参与 WGMMMA 和写回)
2823 //
2824 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2825 // 的延迟被计算完全掩盖。
2826 //
2827 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2828 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2829 // Block: (256, 1, 1), 2 warpgroups
2830 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2831 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2832           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2833           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2834           const bool BLOCK_SWIZZLE = false>

```

```

2835 __global__ void __launch_bounds__(NUM_THREADS)
2836     hgemm_wgmma_stages_tn(
2837         int M, int N, int K, half *C,
2838         const CUtensorMap *__restrict__ tensorMapA,
2839         const CUtensorMap *__restrict__ tensorMapB) {
2840     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2841     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2842     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2843     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2844     // 不展开宿主侧 create_tensor_map 细节。
2845
2846     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2847     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2848     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2849     const int by = blockIdx.y;
2850     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2851     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2852     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2853     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2854     // 当只有一个 consumer 时, 它负责全部 BM=128 行
2855     constexpr int B_WG_M = BM / num_consumers; // 128
2856
2857     // 边界检查: 确保当前 block 不超出 M/N 范围
2858     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2859         return;
2860
2861     // ---- Shared Memory 分配 ----
2862     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2863     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2864     extern __shared__ __align__(128) uint8_t smem_AB[];
2865     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2866         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem_AB);
2867     half *s_a = s.A;
2868     half *s_b = s.B;
2869
2870     // ---- cuda::barrier 初始化 ----
2871     //
2872     // ★ Barrier 机制速查 (arrive / wait 核心语义):
2873     //
2874     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
2875     //
2876     //   init(bar, N): 设置 arrive_count = N。
2877     //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2878     //
2879     //   arrive(): 线程声明"我已到达此 barrier 点"。
2880     //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
2881     //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
2882     //
2883     //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
2884     //       - 即: 等待 phase 翻转。
2885     //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
2886     //                       (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
2887     //
2888     //   典型用法: b.wait(b.arrive())
2889     //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
2890     //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2891     //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2892     //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
2893     //
2894     // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129):
2895     //
2896     //   Producer (1 thread)           Consumer (128 threads)
2897     //   _____                     _____

```

```

2898 // C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2899 // ┌ for each k_tile: ┐ ┌ for each k_tile: ┐
2900 // | P1: arrive+wait(empty[q]) | C1: arrive+wait(full[q])
2901 // |   ↓ 等 stage q 被消费完 |   ↓ 等 TMA 数据就绪
2902 // | P2: TMA(A[q]) + TMA(B[q]) | C2: WGMMMA 计算
2903 // |   ↓ 异步拷贝 | C3: wait WGMMMA 完成
2904 // | P3: arrive_tx(full[q]) | C4: arrive(empty[q])
2905 // |   ↑ 通知: stage q 数据就绪 |   ↑ 通知: stage q 已消费完
2906 // └──────────────────┘ └──────────────────┘
2907 //
2908 // 关键不变式 (invariant) :
2909 //   - Producer 不会覆盖 Consumer 正在读的 stage
2910 //   - Consumer 不会读 Producer 还没写完的 stage
2911 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
2912 //
2913 // 每个 stage 有两个 barrier:
2914 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
2915 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
2916 //
2917 // arrive_count = 128 (consumer) + 1 (producer) = 129:
2918 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
2919 #pragma nv_diag_suppress static_var_with_dynamic_init
2920 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
2921 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
2922 #pragma nv_diag_default static_var_with_dynamic_init
2923
2924 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
2925 const int num_blocks_k = K / BK;
2926 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
2927 const int tid = threadIdx.x % 128; // 0~127 within warpgroup
2928
2929 // 初始化 barriers (仅 thread 0 执行)
2930 if (threadIdx.x == 0) {
2931     for (int i = 0; i < K_STAGE; ++i) {
2932         // arrive_count = 129: 128 consumer + 1 producer
2933         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
2934         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
2935         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
2936         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
2937         init(&empty[i], num_consumers * 128 + 1); // same
2938     }
2939     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
2940     // 对 async proxy (TMA/WGMMMA) 可见。
2941     cuda::device::experimental::fence_proxy_async_shared_cta();
2942 }
2943 __syncthreads();
2944
2945 // =====
2946 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
2947 // =====
2948 //
2949 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
2950 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
2951 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
2952 //
2953 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
2954 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
2955 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
2956 //
2957 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
2958 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
2959 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
2960 //

```

```

2961 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
2962 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
2963 // =====
2964 // Producer Warpgroup (WG0, threadIdx.x 0~127)
2965 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
2966 // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
2967 // =====
2968 if (wg_idx == 0) {
2969     if (tid == 0) {
2970         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
2971         int qidx = 0;
2972         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2973             if (qidx == K_STAGE)
2974                 qidx = 0;
2975
2976             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
2977             //
2978             // 模式 empty[qidx].wait(empty[qidx].arrive()):
2979             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
2980             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
2981             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
2982             //     → phase 立即翻转。
2983             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
2984             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
2985             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
2986             //
2987             // 语义：Producer 说"我准备好写 stage qidx 了，Consumer 用完没有？"
2988             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
2989             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
2990             empty[qidx].wait(empty[qidx].arrive());
2991
2992             // Step P2: 提交 TMA 2D 拷贝指令
2993             // cp_async_bulk_tensor_2d_global_to_shared:
2994             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
2995             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
2996             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
2997             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
2998             //
2999             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
3000             // 无需线程逐元素搬运，零寄存器开销。
3001             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3002             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3003                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
3004                 full[qidx]);
3005
3006             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3007             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3008                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
3009                 full[qidx]);
3010
3011             // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
3012             //
3013             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3014             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3015             //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3016             //   - Phase 翻转条件：
3017             //     (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3018             //     (b) 所有声明的 async 字节已写入 smem
3019             //   两个条件都满足后 phase 才翻转，Consumer 的 wait() 才返回。
3020             //   - 即：Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3021             [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
3022                 full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
3023         }

```

```

3024 }
3025 }
3026 // =====
3027 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3028 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3029 // 所有 128 个线程 (4 warps) 全部参与。
3030 // =====
3031 else {
3032     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3033     //
3034     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3035     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
3036     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3037     //
3038     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3039     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
3040     for (int i = 0; i < K_STAGE; ++i) {
3041         [[maybe_unused]] auto token = empty[i].arrive();
3042     }
3043
3044     // 累加器寄存器声明
3045     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
3046     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3047     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3048     //   - d[*][g][*]: N 方向第 g 组 16 列
3049     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3050     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3051     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3052     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
3053
3054     int qidx = 0;
3055     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3056     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3057         if (qidx == K_STAGE)
3058             qidx = 0;
3059
3060         // Step C1: 等待 TMA 数据就绪 (full 信号)
3061         //
3062         // 模式 full[qidx].wait(full[qidx].arrive()):
3063         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3064         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3065         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3066         //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3067         //
3068         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有?"
3069         full[qidx].wait(full[qidx].arrive());
3070
3071         // Step C2: 发射 WGMMMA 指令序列
3072         //
3073         // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3074         // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3075         //
3076         // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3077         //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3078         //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3079         //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3080         //
3081         // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3082         //   D[64x128] += A[64x16] × B[16x128]
3083         //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3084         //   ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都用 ScaleD=1。
3085         WGMMMA_FENCE();
3086

```

```

3087 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
3088 // 每个 atom 处理 64 行 M 方向数据。
3089 #pragma unroll
3090 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
3091     // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
3092     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
3093     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
3094
3095     // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
3096     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3097 #pragma unroll
3098     for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
3099         // 第 k_it 次 WGMMMA:
3100         //   A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
3101         //   B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
3102         //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3103         //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3104         //   ScaleD=1: 累加 (K 维迭代需要累积结果)
3105         WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
3106                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
3107                                     1, // ScaleD=1: accumulate (不清零)
3108                                     1, 1, 0, 0);
3109     }
3110 }
3111
3112 // Step C3: 提交并等待 WGMMMA 完成
3113 //
3114 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3115 //   将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3116 //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3117 //
3118 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3119 //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3120 //   * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3121 //   两者都是 "wait until only N or fewer of the most recent groups are pending"。
3122 //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3123 //   必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3124 WGMMMA_COMMIT_GROUP();
3125 WGMMMA_WAIT_GROUP(0);
3126
3127 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
3128 //
3129 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
3130 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3131 //
3132 // 语义: Consumer 说 "stage qidx 我已经读完了, 你可以放心覆盖了。"
3133 [[maybe_unused]] auto token = empty[qidx].arrive();
3134 }
3135
3136 // =====
3137 // Epilogue: 将寄存器中的累加结果写回 global memory C
3138 // =====
3139 //
3140 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3141 //
3142 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3143 // 这些 half 分布在 d[2][8][4] 数组中:
3144 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3145 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3146 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3147 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3148 //
3149 // 其中:

```

```

3150 // row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
3151 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3152 //
3153 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3154 // +-----+-----+
3155 // | reg[0] | reg[2] | <- rows [row, row+8)
3156 // |(col,+2)|(col+8)|
3157 // +-----+-----+
3158 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3159 // |(col,+2)|(col+8)|
3160 // +-----+-----+
3161 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3162 //
3163 // 三维遍历:
3164 // m_it=0: rows 0~63, m_it=1: rows 64~127
3165 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3166 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3167 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3168
3169 const int lane = tid % 32;
3170 const int warp = tid / 32;
3171 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3172 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3173 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3174 //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3175 //         因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3176 //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3177 const int row = warp * 16 + lane / 4;
3178 // block_C: 当前 C tile 在 global memory 中的起始地址
3179 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3180 half *block_C = C + by * BM * N + bx * BN;
3181
3182 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3183 #pragma unroll
3184 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
3185     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
3186     #pragma unroll
3187     for (int g = 0; g < WGMMMA_N / 16; ++g) {
3188         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3189         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3190         // 左上象限: (row, col)
3191         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
3192             d[m_it][g][0];
3193         // 左下象限: (row+8, col)
3194         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
3195             d[m_it][g][1];
3196         // 右上象限: (row, col+8)
3197         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
3198             d[m_it][g][2];
3199         // 右下象限: (row+8, col+8)
3200         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
3201             d[m_it][g][3];
3202     }
3203 }
3204 }
3205 }
3206
3207 #if (defined(NOTES_V2_ENABLE_WGMMMA))
3208 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3209 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3210 // - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3211 //   以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3212 // - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!

```

```

3213 // 对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3214 // - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3215 // - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3216 // - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3217 //
3218 // ★ gmem_prob_stride + 1 的含义:
3219 // 语法层面 — 数组名退化 + 指针算术:
3220 // gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3221 // (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3222 // gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3223 // 语义层面 — 跳过隐式的最内维 stride:
3224 // cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3225 // stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3226 // 固定等于 sizeof(element_type), 不需要显式传入。
3227 // 对于 tensorRank=2 的 FP16 矩阵:
3228 // dim 0 (内维, K) : stride = sizeof(half) ← 隐式, API 不需要
3229 // dim 1 (外维, M) : stride = sizeof(half)*K ← 需要传给 API
3230 // gmem_prob_stride 数组的布局是内维在前:
3231 // [0] = sizeof(half) ← 内维, 不传
3232 // [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3233 // 所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3234 //
3235 // ★ smem_box_stride 为什么不需要 +1?
3236 // 与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3237 // 最内维 stride 也必须显式传入 (不隐式)。
3238 // smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3239 // 都是连续存放的, 维度间没有 padding/stride。
3240 // 对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3241 // 即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3242 // 所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3243 // 两个 stride 值, 不需要跳过任何一个。
3244 //
3245 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3246
3247 template <int BlockMajorSize, int BlockMinorSize>
3248 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3249 half *gmem_ptr,
3250 int blocks_height,
3251 int blocks_width) {
3252 void *gmem_address = (void *)gmem_ptr;
3253 uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3254 (uint64_t)BlockMajorSize * blocks_height,
3255 1, 1, 1};
3256 uint64_t gmem_prob_stride[5] = {
3257 sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3258 uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3259 uint32_t(BlockMajorSize), 1, 1, 1};
3260 uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3261 CUresult result = cuTensorMapEncodeTiled(
3262 tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3263 gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3264 CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3265 CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3266 if (result != CUDA_SUCCESS)
3267 printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3268 }
3269
3270 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
3271 half *src, int blocks_height, int blocks_width) {
3272 CUtensorMap *tma_map_d;
3273 cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
3274 CUtensorMap tma_map_host;
3275 create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);

```

```

3276     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
3277               cudaMemcpyHostToDevice);
3278     return tma_map_d;
3279 }
3280 #endif /* NOTES_V2_ENABLE_WGMMA */
3281
3282 // =====
3283 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3284 // =====
3285 // 面试要点 (FlashAttention 算法) :
3286 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3287 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3288 // 2. FA 三板斧:
3289 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3290 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3291 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3292 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3293 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3294 //    - 优点: 减少 warp 间通信和 shuffle
3295 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3296 //    for each K,V tile:
3297 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3298 //        m_new = max(m_old, row_max(S_cur * scale))
3299 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3300 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3301 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3302 //    O_final = O_new / l_final
3303 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3304 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3305 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3306 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3307 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3308 //      都天然同构”的通用结论
3309 //
3310 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3311 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3312 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3313 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3314 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3315
3316 // ---- 寄存器填充辅助函数 ----
3317 template <typename T, int M, const int N, const int K = 2>
3318 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3319     #pragma unroll
3320     for (int i = 0; i < M; ++i)
3321         #pragma unroll
3322         for (int j = 0; j < N; ++j)
3323             #pragma unroll
3324             for (int k = 0; k < K; ++k)
3325                 R[i][j][k] = val;
3326 }
3327
3328 template <typename T, int M, const int N = 2>
3329 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3330     #pragma unroll
3331     for (int i = 0; i < M; ++i)
3332         #pragma unroll
3333         for (int j = 0; j < N; ++j)
3334             R[i][j] = val;
3335 }
3336
3337 // =====
3338 // FlashAttention-2 Split-Q Kernel (完整实现)

```

```

3339 // =====
3340 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3341 //
3342 // Tile 设计 (以 kHeadDim=64 为例) :
3343 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3344 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3345 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3346 //
3347 // 执行流程:
3348 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3349 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3350 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3351 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3352 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3353 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3354 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3355 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3356 //   7)   3e: Online rescaling — Q_new = exp(m_old-m_new)*Q_old + P@V
3357 //   8) 最终 rescale: Q_final = (1/l_final) * Q_final
3358 //   9) Epilogue: warp shuffle + 128-bit collective store
3359
3360 template <
3361     const int kHeadDim,          // head dim: 32, 64, 128
3362     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
3363     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
3364     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
3365     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
3366     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
3367     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3368     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
3369     const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
3370     const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
3371     const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
3372     const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3373     const int kStage,            // pipeline stages for K: 1 or 2
3374     const int kPad>              // padding for bank conflict avoidance
3375 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3376     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3377                                     int QKV_seqlen, int QKV_head) {
3378     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3379     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3380     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3381     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3382     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3383     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3384     const float scale = 1.0f / sqrtf((float)kHeadDim);
3385
3386     // Block indexing
3387     const int QKV_batch_id = blockIdx.y / QKV_head;
3388     const int QKV_head_id = blockIdx.y % QKV_head;
3389     const int Q_tile_id = blockIdx.x;
3390     const int tid = threadIdx.x;
3391     const int warp_id = tid / WARP_SIZE;
3392     const int lane_id = tid % WARP_SIZE;
3393     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3394     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3395
3396     // Global memory base offsets for this (batch, head)
3397     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3398     const int Q_gmem_offset =
3399         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3400         kHeadDim;
3401     const int K_gmem_offset = Q_gmem_offset;

```

```

3402 const int V_gmem_offset = Q_gmem_offset;
3403 const int O_gmem_offset = Q_gmem_offset;
3404
3405 // Thread-to-smem mapping for cooperative load
3406 int load_smem_Q_Br = tid / (kNumThreads / Br);
3407 int load_smem_Q_d =
3408     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3409 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3410 int load_smem_K_d =
3411     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3412 int load_smem_V_Bc = tid / (kNumThreads / Bc);
3413 int load_smem_V_d =
3414     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3415
3416 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3417 if (load_gmem_Q_Br >= QKV_seqlen)
3418     return;
3419
3420 // ---- Shared memory layout ----
3421 extern __shared__ half smem[];
3422 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3423 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3424 half *Q_tile_smem = smem;
3425 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3426 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3427 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3428 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3429
3430 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3431 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3432 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3433
3434 // ---- Online Softmax persistent state ----
3435 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3436 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3437 float lane_block_row_max_old[kValTileSeqLenQ][2];
3438 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3439 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3440 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3441
3442 // ---- Register allocation ----
3443 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3444 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3445 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3446 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3447 // 对单个 m16n8k16 tile 而言:
3448 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3449 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3450 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3451 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3452 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3453 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3454     [2]; // O accumulator (final output)
3455
3456 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3457 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3458
3459 // =====
3460 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3461 // =====
3462 {
3463     int load_gmem_Q_addr =
3464         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;

```

```

3465     uint32_t load_smem_Q_ptr =
3466         smem_Q_base_ptr +
3467         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3468 #pragma unroll
3469     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3470         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3471     }
3472     CP_ASYNC_COMMIT_GROUP();
3473 }
3474
3475 // =====
3476 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3477 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3478 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
3479 // =====
3480 if constexpr (kStage > 1) {
3481 #pragma unroll
3482     for (int stage = 0; stage < (kStage - 1); ++stage) {
3483         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3484         int load_gmem_K_addr =
3485             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3486         uint32_t load_smem_K_ptr =
3487             smem_K_base_ptr +
3488             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3489              load_smem_K_d) *
3490             sizeof(half);
3491 #pragma unroll
3492         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3493             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3494         }
3495         CP_ASYNC_COMMIT_GROUP();
3496     }
3497     CP_ASYNC_WAIT_GROUP(kStage - 2);
3498     __syncthreads();
3499 }
3500
3501 // =====
3502 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3503 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3504 // =====
3505 #pragma unroll 1
3506 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3507     int smem_sel = tile_K_seqlen % kStage;
3508     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3509
3510     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3511     if constexpr (kStage > 1) {
3512         // 只有 kStage>1 才能真正做 K 的 pipeline:
3513         // smem_sel 负责“当前正在计算”的 K tile, smem_sel_next 负责“下一轮预取”的 K tile。
3514         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3515         // Load current V tile (no pipeline for V — one stage is enough)
3516         {
3517             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3518             int load_gmem_V_addr =
3519                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3520             uint32_t load_smem_V_ptr =
3521                 smem_V_base_ptr +
3522                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3523 #pragma unroll
3524             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3525                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3526             }
3527             CP_ASYNC_COMMIT_GROUP();

```

```

3528     }
3529
3530     // Prefetch next K tile (pipelined)
3531     if ((tile_K_seqlen + 1) < Tc) {
3532         int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3533         int load_gmem_K_addr =
3534             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3535         uint32_t load_smem_K_ptr =
3536             smem_K_base_ptr +
3537             (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3538              load_smem_K_d) *
3539             sizeof(half);
3540     #pragma unroll
3541         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3542             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3543         }
3544         CP_ASYNC_COMMIT_GROUP();
3545     }
3546 }
3547
3548 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3549 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3550 #pragma unroll
3551 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3552     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3553     #pragma unroll
3554     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3555         int warp_smem_Q_Br =
3556             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3557         int lane_smem_Q_Br =
3558             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3559         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3560         uint32_t lane_smem_Q_ptr =
3561             smem_Q_base_ptr +
3562             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3563         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3564                    lane_smem_Q_ptr);
3565     }
3566
3567     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3568     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3569     #pragma unroll
3570     for (int j = 0; j < kValTileSeqLenK; ++j) {
3571         int warp_smem_K_Bc =
3572             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3573         int lane_smem_K_Bc =
3574             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3575         int lane_smem_K_d =
3576             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3577         uint32_t lane_smem_K_ptr =
3578             smem_K_base_ptr +
3579             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3580              lane_smem_K_d) *
3581             sizeof(half);
3582         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3583     }
3584
3585     // MMA: S[tile] += Q[tile] @ K^T[tile]
3586     #pragma unroll
3587     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3588         #pragma unroll
3589         for (int j = 0; j < kValTileSeqLenK; ++j) {
3590             HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],

```

```

3591         R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3592         R_S[i][j][1]);
3593     }
3594 }
3595 } // end loop over d
3596 __syncthreads();
3597
3598 // =====
3599 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3600 // =====
3601 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3602 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3603 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3604 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3605 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3606 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3607 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3608 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3609 // kValTileSeqLenK tiles.
3610
3611 float lane_row_max_new[kValTileSeqLenQ][2];
3612 float lane_row_sum_new[kValTileSeqLenQ][2];
3613 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3614 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3615
3616 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3617 #pragma unroll
3618 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3619     #pragma unroll
3620     for (int j = 0; j < kValTileSeqLenK; ++j) {
3621         // Extract half values from R_S registers (C matrix fragment layout)
3622         float2 t_reg_S_0 =
3623             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3624         float2 t_reg_S_1 =
3625             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3626         // S = (Q@K^T) * scale
3627         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3628         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3629         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3630         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3631     }
3632     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3633     // {0,4,8,...,28} hold valid data)
3634     lane_row_max_new[i][0] =
3635         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3636     lane_row_max_new[i][1] =
3637         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3638 }
3639
3640 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3641 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3642 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3643 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3644 #pragma unroll
3645 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3646     // m_new = max(m_old, m_cur)
3647     float block_row_max_new_0 =
3648         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3649     float block_row_max_new_1 =
3650         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3651
3652     #pragma unroll
3653     for (int j = 0; j < kValTileSeqLenK; ++j) {

```

```

3654 float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3655 float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3656
3657 // P = exp(S * scale - m_new), 用 fma 保证精度
3658 t_reg_S_0.x =
3659     __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3660 t_reg_S_0.y =
3661     __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3662 t_reg_S_1.x =
3663     __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3664 t_reg_S_1.y =
3665     __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3666
3667 // Accumulate row sums
3668 lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3669 lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3670
3671 // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3672 HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3673 HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3674 }
3675
3676 // Warp-level reduce sum (warp_size = 4, same as max)
3677 lane_row_sum_new[i][0] =
3678     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3679 lane_row_sum_new[i][1] =
3680     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3681 }
3682 __syncthreads();
3683
3684 // =====
3685 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3686 // =====
3687 // Wait for V to be ready before computing P@V
3688 if constexpr (kStage > 1) {
3689     if ((tile_K_seqLen + 1) < Tc) {
3690         CP_ASYNC_WAIT_GROUP(1);
3691     } else {
3692         CP_ASYNC_WAIT_GROUP(0);
3693     }
3694 } else {
3695     CP_ASYNC_WAIT_GROUP(0);
3696 }
3697 __syncthreads();
3698
3699 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3700
3701 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3702 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3703 #pragma unroll
3704 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3705     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3706     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3707     // transposed ldmatrix
3708 #pragma unroll
3709 for (int j = 0; j < kValTileHeadDimV; ++j) {
3710     int warp_smem_V_d =
3711         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3712     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3713     int lane_smem_V_d = warp_smem_V_d;
3714     uint32_t lane_smem_V_ptr =
3715         smem_V_base_ptr +
3716         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);

```

```

3717     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3718 }
3719
3720 // P matrix layout in R_S[i][j][2]:
3721 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3722 //   | 64x64 | warp_KV 0 |
3723 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3724 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3725 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3726 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3727 // tile_V_Bc selects which 16-column slice of P to use:
3728 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3729 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3730 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3731 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3732 // 对应的 MMA A fragment 布局可以这样记:
3733 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3734 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3735 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3736 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3737 // fragment 都无条件成立的通用结论。layout转换逻辑:
3738 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3739 int w = tile_V_Bc * 2;
3740 #pragma unroll
3741 for (int i = 0; i < kValTileSeqLenP; ++i) {
3742 #pragma unroll
3743     for (int j = 0; j < kValTileHeadDimV; ++j) {
3744         HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
3745                 R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3746                 R_V[j][0], R_V[j][1], // B fragment = V
3747                 R_O[i][j][0], R_O[i][j][1]); // C fragment output
3748     }
3749 }
3750 } // end for tile_V_Bc
3751 __syncthreads();
3752
3753 // =====
3754 // 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$ 
3755 // =====
3756 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{\text{old}} - m_{\text{new}})$  做 0 与 1 的 rescale
3757 #pragma unroll
3758 for (int i = 0; i < kValTileSeqLenP; ++i) {
3759     float block_row_max_new_0 = lane_row_max_new[i][0];
3760     float block_row_max_new_1 = lane_row_max_new[i][1];
3761     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3762     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3763
3764     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3765     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3766
3767     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3768     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3769
3770     // Handle first iteration: m_old = -inf, need to use m_new directly
3771     block_row_max_old_0 =
3772         (tile_K_seqLen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3773     block_row_max_old_1 =
3774         (tile_K_seqLen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3775
3776     float rescale_o_factor_0 =
3777         __expf(block_row_max_old_0 - block_row_max_new_0);
3778     float rescale_o_factor_1 =
3779         __expf(block_row_max_old_1 - block_row_max_new_1);

```

```

3780
3781 // Rescale O_old + Add P@V in one fused step
3782 #pragma unroll
3783 for (int j = 0; j < kValTileHeadDimV; ++j) {
3784     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3785     // reg[0] -> rows 0~7 的 {c0,c1}
3786     // reg[1] -> rows 8~15 的 {c2,c3}
3787     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3788     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3789     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3790     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3791
3792     // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
3793     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3794     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3795     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3796     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3797
3798     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3799     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3800 }
3801
3802 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3803 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3804 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3805 lane_block_row_sum_old[i][0] = __fmaf_rn(
3806     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3807 lane_block_row_sum_old[i][1] = __fmaf_rn(
3808     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3809
3810 // Update m
3811 lane_block_row_max_old[i][0] = block_row_max_new_0;
3812 lane_block_row_max_old[i][1] = block_row_max_new_1;
3813 }
3814
3815 // Wait for next K tile to be ready in smem before next iteration
3816 if constexpr (kStage > 1) {
3817     if ((tile_K_seqlen + 1) < Tc) {
3818         CP_ASYNC_WAIT_GROUP(0);
3819     }
3820     __syncthreads();
3821 }
3822 } // end outer loop over K seqlen
3823 __syncthreads();
3824
3825 // =====
3826 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
3827 // =====
3828 #pragma unroll
3829 for (int i = 0; i < kValTileSeqLenP; ++i) {
3830     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3831     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
3832 #pragma unroll
3833 for (int j = 0; j < kValTileHeadDimV; ++j) {
3834     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3835     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3836     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3837     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3838     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3839     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3840     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3841     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3842 }

```

```

3843 }
3844
3845 // =====
3846 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3847 // =====
3848 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3849 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3850 #pragma unroll
3851 for (int i = 0; i < kValTileSeqLenP; ++i) {
3852 #pragma unroll
3853     for (int j = 0; j < kValTileHeadDimV; ++j) {
3854         uint32_t R_Z[2][4];
3855         R_Z[0][0] = R_D[i][j][0];
3856         R_Z[1][0] = R_D[i][j][1];
3857         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3858         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3859         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3860         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3861         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3862         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3863
3864         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3865         if (lane_id % 4 == 0) {
3866             int store_warp_regs_0_Br =
3867                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3868             int store_lane_gmem_0_Br =
3869                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3870             int store_warp_regs_0_d =
3871                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3872             int store_lane_gmem_0_d = store_warp_regs_0_d;
3873             int store_gmem_0_addr_0 = 0_gmem_offset +
3874                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3875                 store_lane_gmem_0_d;
3876             int store_gmem_0_addr_1 = 0_gmem_offset +
3877                 (store_lane_gmem_0_Br + 8) * kHeadDim +
3878                 store_lane_gmem_0_d;
3879             // LDST128BITS = reinterpret_cast<float4*>
3880             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
3881                 *reinterpret_cast<float4*>(&R_Z[0][0]);
3882             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
3883                 *reinterpret_cast<float4*>(&R_Z[1][0]);
3884         }
3885     }
3886 }
3887 }
3888
3889 // =====
3890 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3891 // =====
3892
3893 static inline void check(cudaError_t err, const char *msg) {
3894     if (err != cudaSuccess) {
3895         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3896         exit(EXIT_FAILURE);
3897     }
3898 }
3899
3900 static void test_block_reduce(int N) {
3901     srand(42);
3902     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3903     for (int i = 0; i < N; i++)

```

```

3906     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3907
3908     // CPU reference: sum of all elements
3909     double ref = 0.0;
3910     for (int i = 0; i < N; i++) ref += (double)h_a[i];
3911
3912     float *d_a, *d_y;
3913     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
3914     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
3915
3916     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
3917     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
3918
3919     dim3 block(128);
3920     dim3 grid((N + 127) / 128);
3921     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
3922     check(cudaGetLastError(), "blockreduce launch");
3923     check(cudaDeviceSynchronize(), "blockreduce sync");
3924
3925     float result;
3926     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
3927
3928     float err = fabsf(result - (float)ref);
3929     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
3930           err < 1e-2f ? "PASS" : "FAIL");
3931
3932     free(h_a);
3933     cudaFree(d_a); cudaFree(d_y);
3934 }
3935
3936
3937 static void test_dot(int N) {
3938
3939     srand(42);
3940     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3941     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3942     for (int i = 0; i < N; i++) {
3943         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3944         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3945     }
3946
3947     // CPU reference
3948     double ref = 0.0;
3949     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
3950
3951     float *d_a, *d_b, *d_y;
3952     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
3953     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
3954     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
3955
3956     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
3957     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
3958     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
3959
3960     dim3 block(128);
3961     dim3 grid((N + 127) / 128);
3962     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
3963     check(cudaGetLastError(), "dot launch");
3964     check(cudaDeviceSynchronize(), "dot sync");
3965
3966     float result;
3967     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
3968

```

```

3969 float err = fabsf(result - (float)ref);
3970 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
3971     err < 1e-2f ? "PASS" : "FAIL");
3972
3973 // ---- Dot Vec4 ----
3974 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
3975 dim3 block_v4(32);
3976 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
3977 check(cudaGetLastError(), "dot_vec4 launch");
3978 check(cudaDeviceSynchronize(), "dot_vec4 sync");
3979
3980 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
3981 float err_v4 = fabsf(result - (float)ref);
3982 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
3983
3984 free(h_a); free(h_b);
3985 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
3986 }
3987
3988
3989 static void test_relu(int N) {
3990     srand(42);
3991     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3992     float *h_y = (float *)malloc((size_t)N * sizeof(float));
3993     for (int i = 0; i < N; i++)
3994         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3995
3996     float *d_x, *d_y;
3997     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
3998     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
3999     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4000
4001     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4002     dim3 block256(256);
4003     dim3 grid256((N + 255) / 256);
4004     relu<<<grid256, block256>>>(d_x, d_y, N);
4005     check(cudaGetLastError(), "relu launch");
4006     check(cudaDeviceSynchronize(), "relu sync");
4007     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4008     float max_err = 0.0f;
4009     for (int i = 0; i < N; i++) {
4010         float expected = fmaxf(0.0f, h_x[i]);
4011         float err = fabsf(h_y[i] - expected);
4012         if (err > max_err) max_err = err;
4013     }
4014     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4015
4016     dim3 block64(64);
4017     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4018     check(cudaGetLastError(), "relu_vec4 launch");
4019     check(cudaDeviceSynchronize(), "relu_vec4 sync");
4020     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4021     max_err = 0.0f;
4022     for (int i = 0; i < N; i++) {
4023         float expected = fmaxf(0.0f, h_x[i]);
4024         float err = fabsf(h_y[i] - expected);
4025         if (err > max_err) max_err = err;
4026     }
4027     printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4028
4029     free(h_x); free(h_y);
4030     cudaFree(d_x); cudaFree(d_y);
4031 }

```

```

4032
4033
4034 static void test_elementwise(int N) {
4035     srand(42);
4036     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4037     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4038     for (int i = 0; i < N; i++) {
4039         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4040         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4041     }
4042
4043     float *d_a, *d_b, *d_c;
4044     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4045     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4046     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4047     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4048     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4049
4050     dim3 block256(256);
4051     dim3 grid256((N + 255) / 256);
4052     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4053     check(cudaGetLastError(), "eadd launch");
4054     check(cudaDeviceSynchronize(), "eadd sync");
4055     float *h_c = (float *)malloc((size_t)N * sizeof(float));
4056     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4057     float max_err = 0.0f;
4058     for (int i = 0; i < N; i++) {
4059         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4060         if (err > max_err) max_err = err;
4061     }
4062     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4063
4064     dim3 block64(64);
4065     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4066     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4067     check(cudaGetLastError(), "eadd_vec4 launch");
4068     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4069     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4070     max_err = 0.0f;
4071     for (int i = 0; i < N; i++) {
4072         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4073         if (err > max_err) max_err = err;
4074     }
4075     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4076
4077     free(h_a); free(h_b); free(h_c);
4078     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4079 }
4080
4081
4082 static void test_histogram(int N) {
4083     srand(42);
4084     int BINS = 16;
4085     int *h_hist = (int *)calloc(BINS, sizeof(int));
4086     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4087     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4088     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4089     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4090
4091     int *d_idx, *d_hist;
4092     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4093     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4094     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");

```

```

4095 check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4096
4097 dim3 block256(256);
4098 dim3 grid256((N + 255) / 256);
4099 histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4100 check(cudaGetLastError(), "histogram launch");
4101 check(cudaDeviceSynchronize(), "histogram sync");
4102 check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4103
4104 float max_err = 0.0f;
4105 for (int i = 0; i < BINS; i++) {
4106     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4107     if (err > max_err) max_err = err;
4108 }
4109 printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4110
4111 free(h_hist); free(h_hist_ref); free(h_idx);
4112 cudaFree(d_idx); cudaFree(d_hist);
4113 }
4114
4115
4116 static void test_merge_attn_states(int num_tokens, int num_heads,
4117                                   int head_size) {
4118
4119     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4120     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4121
4122     float *h_prefix_out = (float *)malloc(out_size);
4123     float *h_suffix_out = (float *)malloc(out_size);
4124     float *h_prefix_lse = (float *)malloc(lse_size);
4125     float *h_suffix_lse = (float *)malloc(lse_size);
4126     float *h_output_ref = (float *)malloc(out_size);
4127
4128     srand(42);
4129     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4130         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4131         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4132     }
4133     for (int i = 0; i < num_heads * num_tokens; i++) {
4134         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4135         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4136     }
4137
4138     // CPU reference: 与 kernel 完全相同的浮点计算
4139     for (int t = 0; t < num_tokens; t++) {
4140         for (int h = 0; h < num_heads; h++) {
4141             float p_lse = h_prefix_lse[h * num_tokens + t];
4142             float s_lse = h_suffix_lse[h * num_tokens + t];
4143             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4144             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4145
4146             float max_lse = fmaxf(p_lse, s_lse);
4147             p_lse -= max_lse;
4148             s_lse -= max_lse;
4149             float p_se = expf(p_lse);
4150             float s_se = expf(s_lse);
4151             float p_scale = p_se / (p_se + s_se);
4152             float s_scale = s_se / (p_se + s_se);
4153
4154             int head_off = t * num_heads * head_size + h * head_size;
4155             for (int d = 0; d < head_size; d++) {
4156                 h_output_ref[head_off + d] =
4157                     h_prefix_out[head_off + d] * p_scale +

```

```

4158         h_suffix_out[head_off + d] * s_scale;
4159     }
4160 }
4161 }
4162
4163 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4164 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4165 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4166 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4167 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4168 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4169
4170 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4171                 cudaMemcpyHostToDevice),
4172        "merge_attn H2D p_out");
4173 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4174                 cudaMemcpyHostToDevice),
4175        "merge_attn H2D s_out");
4176 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4177                 cudaMemcpyHostToDevice),
4178        "merge_attn H2D p_lse");
4179 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4180                 cudaMemcpyHostToDevice),
4181        "merge_attn H2D s_lse");
4182
4183 int threads_per_head = head_size / 4;
4184 int total_threads = num_tokens * num_heads * threads_per_head;
4185 dim3 block(128);
4186 dim3 grid((total_threads + 127) / 128);
4187 merge_attn_states<<<grid, block>>>(&d_output, &d_prefix_out, &d_prefix_lse, &d_suffix_out, &d_suffix_lse,
4188                                     num_tokens, num_heads, head_size);
4189 check(cudaGetLastError(), "merge_attn launch");
4190 check(cudaDeviceSynchronize(), "merge_attn sync");
4191
4192 float *h_output = (float *)malloc(out_size);
4193 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4194        "merge_attn D2H");
4195
4196 float max_err = 0.0f;
4197 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4198     float err = fabsf(h_output[i] - h_output_ref[i]);
4199     if (err > max_err) max_err = err;
4200 }
4201 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
4202        max_err < 1e-4f ? "PASS" : "FAIL");
4203
4204 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4205 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4206 h_suffix_lse[0] = 0.0f;
4207 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4208                 cudaMemcpyHostToDevice),
4209        "merge_attn H2D inf lse");
4210 merge_attn_states<<<grid, block>>>(&d_output, &d_prefix_out, &d_prefix_lse, &d_suffix_out, &d_suffix_lse,
4211                                     num_tokens, num_heads, head_size);
4212 check(cudaGetLastError(), "merge_attn inf launch");
4213 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4214 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4215        "merge_attn inf D2H");
4216
4217 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4218 float inf_err = 0.0f;

```

```

4221 for (int d = 0; d < head_size; d++) {
4222     float err = fabsf(h_output[d] - h_suffix_out[d]);
4223     if (err > inf_err) inf_err = err;
4224 }
4225 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
4226        inf_err < 1e-4f ? "PASS" : "FAIL");
4227
4228 free(h_prefix_out);
4229 free(h_suffix_out);
4230 free(h_prefix_lse);
4231 free(h_suffix_lse);
4232 free(h_output_ref);
4233 free(h_output);
4234 cudaFree(d_prefix_out);
4235 cudaFree(d_suffix_out);
4236 cudaFree(d_prefix_lse);
4237 cudaFree(d_suffix_lse);
4238 cudaFree(d_output);
4239 }
4240
4241
4242 static void test_softmax(int N) {
4243     // Use N = blockDim.x so one block processes all elements as one token.
4244     constexpr int NUM_THREADS = 256;
4245     if (N != NUM_THREADS) {
4246         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
4247         return;
4248     }
4249
4250     srand(42);
4251     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4252     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4253     for (int i = 0; i < N; i++)
4254         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4255
4256     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4257     float max_val = -FLT_MAX;
4258     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4259     double sum_exp = 0.0;
4260     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4261     for (int i = 0; i < N; i++)
4262         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4263
4264     float *d_x, *d_y;
4265     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4266     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4267
4268     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4269
4270     dim3 block(256);
4271     dim3 grid(1); // one token covering all N elements
4272     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4273     check(cudaGetLastError(), "softmax launch");
4274     check(cudaDeviceSynchronize(), "softmax sync");
4275
4276     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4277     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4278
4279     float max_err = 0.0f;
4280     for (int i = 0; i < N; i++) {
4281         float err = fabsf(h_y[i] - h_y_ref[i]);
4282         if (err > max_err) max_err = err;
4283     }

```

```

4284 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4285
4286 // ---- Safe Softmax ----
4287 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4288 check(cudaGetLastError(), "safe_softmax launch");
4289 check(cudaDeviceSynchronize(), "safe_softmax sync");
4290 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4291 max_err = 0.0f;
4292 for (int i = 0; i < N; i++) {
4293     float err = fabsf(h_y[i] - h_y_ref[i]);
4294     if (err > max_err) max_err = err;
4295 }
4296 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4297
4298 // ---- Naive Softmax ----
4299 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4300 check(cudaGetLastError(), "naive_softmax launch");
4301 check(cudaDeviceSynchronize(), "naive_softmax sync");
4302 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4303 max_err = 0.0f;
4304 for (int i = 0; i < N; i++) {
4305     float err = fabsf(h_y[i] - h_y_ref[i]);
4306     if (err > max_err) max_err = err;
4307 }
4308 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4309
4310 free(h_x); free(h_y); free(h_y_ref);
4311 cudaFree(d_x); cudaFree(d_y);
4312 }
4313
4314
4315 static void test_rms_norm(int N, int K) {
4316
4317     srand(42);
4318     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4319     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4320     for (int i = 0; i < N * K; i++)
4321         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4322     float g = 1.5f; // gain
4323
4324     // CPU reference: y = (x / rms(x)) * g
4325     float epsilon = 1e-5f;
4326     for (int n = 0; n < N; n++) {
4327         double sum_sq = 0.0;
4328         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4329         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4330         for (int k = 0; k < K; k++)
4331             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4332     }
4333
4334     float *d_x, *d_y;
4335     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4336     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4337     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4338
4339     dim3 block(128);
4340     dim3 grid(N);
4341     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4342     check(cudaGetLastError(), "rmsnorm launch");
4343     check(cudaDeviceSynchronize(), "rmsnorm sync");
4344
4345     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4346     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");

```

```

4347 float max_err = 0.0f;
4348 for (int i = 0; i < N * K; i++) {
4349     float err = fabsf(h_y[i] - h_y_ref[i]);
4350     if (err > max_err) max_err = err;
4351 }
4352 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4353
4354 // ---- RMS Norm Vec4 ----
4355 dim3 block_rv4(32);
4356 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4357 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4358 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4359 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4360 max_err = 0.0f;
4361 for (int i = 0; i < N * K; i++) {
4362     float err = fabsf(h_y[i] - h_y_ref[i]);
4363     if (err > max_err) max_err = err;
4364 }
4365 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4366
4367 free(h_x); free(h_y); free(h_y_ref);
4368 cudaFree(d_x); cudaFree(d_y);
4369 }
4370
4371
4372
4373 static void test_layer_norm(int N, int K) {
4374
4375     srand(42);
4376     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4377     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4378     for (int i = 0; i < N * K; i++)
4379         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4380     float g = 1.5f, b = 0.3f; // gain and bias
4381
4382     // CPU reference: y = ((x - mean) / std) * g + b
4383     float epsilon = 1e-5f;
4384     for (int n = 0; n < N; n++) {
4385         double sum = 0.0;
4386         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4387         float mean = (float)sum / (float)K;
4388         double sum_sq = 0.0;
4389         for (int k = 0; k < K; k++) {
4390             float diff = h_x[n * K + k] - mean;
4391             sum_sq += (double)diff * (double)diff;
4392         }
4393         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4394         for (int k = 0; k < K; k++)
4395             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4396     }
4397
4398     float *d_x, *d_y;
4399     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4400     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4401     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4402
4403     dim3 block(128);
4404     dim3 grid(N);
4405     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4406     check(cudaGetLastError(), "layernorm launch");
4407     check(cudaDeviceSynchronize(), "layernorm sync");
4408
4409     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));

```

```

4410 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4411
4412 float max_err = 0.0f;
4413 for (int i = 0; i < N * K; i++) {
4414     float err = fabsf(h_y[i] - h_y_ref[i]);
4415     if (err > max_err) max_err = err;
4416 }
4417 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4418
4419 // ---- Layer Norm Vec4 ----
4420 dim3 block_lv4(32);
4421 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4422 check(cudaGetLastError(), "layernorm_vec4 launch");
4423 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4424 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4425 max_err = 0.0f;
4426 for (int i = 0; i < N * K; i++) {
4427     float err = fabsf(h_y[i] - h_y_ref[i]);
4428     if (err > max_err) max_err = err;
4429 }
4430 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4431
4432 free(h_x); free(h_y); free(h_y_ref);
4433 cudaFree(d_x); cudaFree(d_y);
4434 }
4435
4436
4437 static void test_rope(int seq_len, int N) {
4438
4439     int total_pairs = seq_len * N;
4440     int total_elems = total_pairs * 2;
4441     size_t size = (size_t)total_elems * sizeof(float);
4442
4443     float *h_x = (float *)malloc(size);
4444     float *h_y_ref = (float *)malloc(size);
4445
4446     srand(42);
4447     for (int i = 0; i < total_elems; i++)
4448         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4449
4450     // CPU reference: 2D rotation for each pair
4451     for (int idx = 0; idx < total_pairs; idx++) {
4452         int token_pos = idx / N;
4453         int token_idx = idx % N;
4454         float x1 = h_x[idx * 2];
4455         float x2 = h_x[idx * 2 + 1];
4456         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4457         float angle = (float)token_pos * theta;
4458         float cos_v = cosf(angle);
4459         float sin_v = sinf(angle);
4460         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4461         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4462     }
4463
4464     float *d_x, *d_y;
4465     check(cudaMalloc(&d_x, size), "rope alloc X");
4466     check(cudaMalloc(&d_y, size), "rope alloc Y");
4467     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4468
4469     dim3 block(256);
4470     dim3 grid((total_pairs + 255) / 256);
4471     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4472     check(cudaGetLastError(), "rope launch");

```

```

4473 check(cudaDeviceSynchronize(), "rope sync");
4474
4475 float *h_y = (float *)malloc(size);
4476 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4477
4478 float max_err = 0.0f;
4479 for (int i = 0; i < total_elems; i++) {
4480     float err = fabsf(h_y[i] - h_y_ref[i]);
4481     if (err > max_err) max_err = err;
4482 }
4483 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4484
4485 free(h_x);
4486 free(h_y);
4487 free(h_y_ref);
4488 cudaFree(d_x);
4489 cudaFree(d_y);
4490 }
4491
4492
4493 static void test_mat_transpose(int row, int col) {
4494
4495     size_t size_in = (size_t)row * col * sizeof(float);
4496     size_t size_out = (size_t)col * row * sizeof(float);
4497
4498     float *h_x = (float *)malloc(size_in);
4499     float *h_y_ref = (float *)malloc(size_out);
4500
4501     srand(42);
4502     for (int i = 0; i < row * col; i++)
4503         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4504
4505     // CPU reference: y[j][i] = x[i][j] (row-major)
4506     for (int i = 0; i < row; i++)
4507         for (int j = 0; j < col; j++)
4508             h_y_ref[j * row + i] = h_x[i * col + j];
4509
4510     float *d_x, *d_y;
4511     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4512     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4513     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4514
4515     dim3 block(16, 16);
4516     dim3 grid((col + 15) / 16, (row + 15) / 16);
4517     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4518     check(cudaGetLastError(), "mattrans launch");
4519     check(cudaDeviceSynchronize(), "mattrans sync");
4520
4521     float *h_y = (float *)malloc(size_out);
4522     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4523
4524     float max_err = 0.0f;
4525     for (int i = 0; i < col * row; i++) {
4526         float err = fabsf(h_y[i] - h_y_ref[i]);
4527         if (err > max_err) max_err = err;
4528     }
4529     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4530
4531     free(h_x);
4532     free(h_y);
4533     free(h_y_ref);
4534     cudaFree(d_x);
4535     cudaFree(d_y);

```

```

4536 }
4537
4538
4539 static void test_mat_transpose_padded(int row, int col) {
4540
4541     size_t size_in = (size_t)row * col * sizeof(float);
4542     size_t size_out = (size_t)col * row * sizeof(float);
4543
4544     float *h_x = (float *)malloc(size_in);
4545     float *h_y_ref = (float *)malloc(size_out);
4546
4547     srand(42);
4548     for (int i = 0; i < row * col; i++)
4549         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4550
4551     // CPU reference: y[j][i] = x[i][j] (row-major)
4552     for (int i = 0; i < row; i++)
4553         for (int j = 0; j < col; j++)
4554             h_y_ref[j * row + i] = h_x[i * col + j];
4555
4556     float *d_x, *d_y;
4557     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4558     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4559     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4560
4561     dim3 block(16, 16);
4562     dim3 grid((col + 15) / 16, (row + 63) / 64);
4563     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4564     check(cudaGetLastError(), "mattrans_padded launch");
4565     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4566
4567     float *h_y = (float *)malloc(size_out);
4568     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4569
4570     float max_err = 0.0f;
4571     for (int i = 0; i < col * row; i++) {
4572         float err = fabsf(h_y[i] - h_y_ref[i]);
4573         if (err > max_err) max_err = err;
4574     }
4575     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4576
4577     free(h_x);
4578     free(h_y);
4579     free(h_y_ref);
4580     cudaFree(d_x);
4581     cudaFree(d_y);
4582 }
4583
4584
4585 static void test_sgemv(int M, int K) {
4586
4587     srand(42);
4588     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4589     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4590     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4591     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4592     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4593
4594     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4595     for (int m = 0; m < M; m++) {
4596         double sum = 0.0;
4597         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4598         h_y_ref[m] = (float)sum;

```

```

4599 }
4600
4601 float *d_a, *d_x, *d_y;
4602 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4603 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4604 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4605
4606 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4607 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4608
4609 dim3 block(32, 4);
4610 dim3 grid((M + 3) / 4);
4611 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4612 check(cudaGetLastError(), "sgemv launch");
4613 check(cudaDeviceSynchronize(), "sgemv sync");
4614
4615 float *h_y = (float *)malloc((size_t)M * sizeof(float));
4616 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4617
4618 float max_err = 0.0f;
4619 for (int m = 0; m < M; m++) {
4620     float err = fabsf(h_y[m] - h_y_ref[m]);
4621     if (err > max_err) max_err = err;
4622 }
4623 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4624
4625 // ---- SGEMV K32 ----
4626 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4627 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4628 check(cudaGetLastError(), "sgemv_k32 launch");
4629 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4630
4631 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4632
4633 max_err = 0.0f;
4634 for (int m = 0; m < M; m++) {
4635     float err = fabsf(h_y[m] - h_y_ref[m]);
4636     if (err > max_err) max_err = err;
4637 }
4638 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4639
4640 // ---- SGEMV K16 ----
4641 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4642 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4643
4644 int K16 = 16;
4645 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4646 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4647 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4648 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4649 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4650
4651 for (int m = 0; m < M; m++) {
4652     double sum = 0.0;
4653     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4654     h_y_ref[m] = (float)sum;
4655 }
4656
4657 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4658 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4659 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4660
4661 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");

```

```

4662 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4663
4664 dim3 grid_k16((M + 7) / 8);
4665 sgemv_k16<<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4666 check(cudaGetLastError(), "sgemv_k16 launch");
4667 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4668
4669 h_y = (float *)malloc((size_t)M * sizeof(float));
4670 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4671
4672 max_err = 0.0f;
4673 for (int m = 0; m < M; m++) {
4674     float err = fabsf(h_y[m] - h_y_ref[m]);
4675     if (err > max_err) max_err = err;
4676 }
4677 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4678
4679 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4680 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4681 }
4682
4683
4684 static void test_sgemm(int M, int N, int K) {
4685
4686     size_t size_a = (size_t)M * K * sizeof(float);
4687     size_t size_b = (size_t)K * N * sizeof(float);
4688     size_t size_c = (size_t)M * N * sizeof(float);
4689
4690     float *h_a = (float *)malloc(size_a);
4691     float *h_b = (float *)malloc(size_b);
4692     float *h_c_ref = (float *)malloc(size_c);
4693
4694     srand(42);
4695     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4696     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4697
4698     float *d_a, *d_b, *d_c;
4699     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4700     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4701     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4702
4703     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4704     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4705
4706     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4707     cublasHandle_t handle;
4708     cublasCreate(&handle);
4709     float alpha = 1.0f, beta = 0.0f;
4710     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4711                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
4712     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4713
4714     // Kernel
4715     cudaEvent_t start, stop;
4716     cudaEventCreate(&start);
4717     cudaEventCreate(&stop);
4718
4719     dim3 block(32, 32);
4720     dim3 grid((N + 31) / 32, (M + 31) / 32);
4721     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4722     check(cudaGetLastError(), "sgemm launch");
4723     check(cudaDeviceSynchronize(), "sgemm sync");
4724

```

```

4725 float *h_c = (float *)malloc(size_c);
4726 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4727
4728 // Verify
4729 float max_err = 0.0f;
4730 for (int i = 0; i < M * N; i++) {
4731     float err = fabsf(h_c[i] - h_c_ref[i]);
4732     if (err > max_err) max_err = err;
4733 }
4734 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4735
4736 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4737
4738 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4739 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4740 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4741 check(cudaGetLastError(), "sgemm_vec4 launch");
4742 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4743
4744 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4745
4746 max_err = 0.0f;
4747 for (int i = 0; i < M * N; i++) {
4748     float err = fabsf(h_c[i] - h_c_ref[i]);
4749     if (err > max_err) max_err = err;
4750 }
4751 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4752
4753 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4754 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4755 cublasDestroy(handle);
4756 cudaEventDestroy(start);
4757 cudaEventDestroy(stop);
4758 }
4759
4760
4761 static void test_hgemm_mma(int M, int N, int K) {
4762
4763     size_t size_a = (size_t)M * K * sizeof(half);
4764     size_t size_b = (size_t)K * N * sizeof(half);
4765     size_t size_c = (size_t)M * N * sizeof(half);
4766
4767     half *h_a = (half *)malloc(size_a);
4768     half *h_b = (half *)malloc(size_b);
4769     half *h_c_ref = (half *)malloc(size_c);
4770
4771     srand(42);
4772     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4773     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4774
4775     // Kernel expects B^T [NxK] row-major layout (TN layout convention).
4776     // We store h_b as B [KxN] row-major for cuBLAS, then create B^T for kernel.
4777     size_t size_b_t = (size_t)N * K * sizeof(half);
4778     half *h_b_t = (half *)malloc(size_b_t);
4779     for (int n = 0; n < N; n++)
4780         for (int k = 0; k < K; k++)
4781             h_b_t[n * K + k] = h_b[k * N + n];
4782
4783     half *d_a, *d_b, *d_b_t, *d_c;
4784     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4785     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4786     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4787     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");

```

```

4788
4789 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4790 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4791 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4792
4793 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4794 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4795 cublasHandle_t handle;
4796 cublasCreate(&handle);
4797 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4798 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4799             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4800             &beta_h, d_c, CUDA_R_16F, N,
4801             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4802 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4803
4804 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4805 cudaEvent_t start, stop;
4806 cudaEventCreate(&start);
4807 cudaEventCreate(&stop);
4808
4809 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4810 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4811 dim3 block(256);
4812 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4813 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4814 check(cudaGetLastError(), "hgemm launch");
4815 check(cudaDeviceSynchronize(), "hgemm sync");
4816
4817 half *h_c = (half *)malloc(size_c);
4818 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4819
4820 // Verify
4821 float max_err = 0.0f;
4822 for (int i = 0; i < M * N; i++) {
4823     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4824     if (err > max_err) max_err = err;
4825 }
4826 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4827
4828 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4829 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4830 cublasDestroy(handle);
4831 cudaEventDestroy(start);
4832 cudaEventDestroy(stop);
4833 }
4834
4835
4836 static void test_hgemm_swizzle(int M, int N, int K) {
4837     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4838     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
4839     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
4840
4841     size_t size_a = (size_t)M * K * sizeof(half);
4842     size_t size_b = (size_t)K * N * sizeof(half);
4843     size_t size_c = (size_t)M * N * sizeof(half);
4844
4845     half *h_a = (half *)malloc(size_a);
4846     half *h_b = (half *)malloc(size_b);
4847     half *h_c_ref = (half *)malloc(size_c);
4848
4849     srand(42);
4850     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

```

```

4851 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4852
4853 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4854 size_t size_b_t = (size_t)N * K * sizeof(half);
4855 half *h_b_t = (half *)malloc(size_b_t);
4856 for (int n = 0; n < N; n++)
4857     for (int k = 0; k < K; k++)
4858         h_b_t[n * K + k] = h_b[k * N + n];
4859
4860 half *d_a, *d_b, *d_b_t, *d_c;
4861 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4862 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4863 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4864 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4865
4866 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4867 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4868 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4869
4870 // cuBLAS FP16 reference
4871 cublasHandle_t handle;
4872 cublasCreate(&handle);
4873 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4874 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4875             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4876             &beta_h, d_c, CUDA_R_16F, N,
4877             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4878 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4879
4880 // MMA swizzle kernel (default params: K_STAGE=3)
4881 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE_S = 3;
4882 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4883 dim3 block(256);
4884 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4885 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4886 check(cudaGetLastError(), "hgemm_swizzle launch");
4887 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4888
4889 half *h_c = (half *)malloc(size_c);
4890 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4891
4892 // Verify
4893 float max_err = 0.0f;
4894 for (int i = 0; i < M * N; i++) {
4895     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4896     if (err > max_err) max_err = err;
4897 }
4898 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA Swizzle", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4899
4900 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4901 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4902 cublasDestroy(handle);
4903 }
4904
4905
4906 #if defined(NOTES_V2_ENABLE_CUTE)
4907 static void test_hgemm_cute(int M, int N, int K) {
4908     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
4909     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
4910     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
4911     // Tile: BM=128, BN=256, BK=32, 128 threads/block
4912
4913     size_t size_a = (size_t)M * K * sizeof(half);

```

```

4914 size_t size_b = (size_t)K * N * sizeof(half);
4915 size_t size_c = (size_t)M * N * sizeof(half);
4916
4917 half *h_a = (half *)malloc(size_a);
4918 half *h_b = (half *)malloc(size_b);
4919 half *h_c_ref = (half *)malloc(size_c);
4920
4921 srand(42);
4922 for (int i = 0; i < M * K; i++)
4923     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4924 for (int i = 0; i < K * N; i++)
4925     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4926
4927 // CuTe kernel expects B^T [N×K] row-major (TN layout).
4928 size_t size_b_t = (size_t)N * K * sizeof(half);
4929 half *h_b_t = (half *)malloc(size_b_t);
4930 for (int n = 0; n < N; n++)
4931     for (int k = 0; k < K; k++)
4932         h_b_t[n * K + k] = h_b[k * N + n];
4933
4934 half *d_a, *d_b, *d_b_t, *d_c;
4935 check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
4936 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
4937 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
4938 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
4939
4940 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
4941 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
4942 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
4943
4944 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4945 cublasHandle_t handle;
4946 cublasCreate(&handle);
4947 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4948 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4949             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4950             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4951 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
4952
4953 // CuTe kernel (Stages=2, TN layout)
4954 launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
4955 check(cudaGetLastError(), "hgemm_cute launch");
4956 check(cudaDeviceSynchronize(), "hgemm_cute sync");
4957
4958 half *h_c = (half *)malloc(size_c);
4959 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
4960
4961 // Verify
4962 float max_err = 0.0f;
4963 for (int i = 0; i < M * N; i++) {
4964     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4965     if (err > max_err) max_err = err;
4966 }
4967 printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4968
4969 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4970 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4971 cublasDestroy(handle);
4972 }
4973 #endif /* NOTES_V2_ENABLE_CUTE */
4974
4975
4976 #if defined(NOTES_V2_ENABLE_WGMMMA)

```

```

4977 static void test_hgemm_wgmma(int M, int N, int K) {
4978     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
4979     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4980     // Kernel: hgemm_wgmma_stages_tn with default template params
4981
4982     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
4983
4984     // M, K must be divisible by tile dims
4985     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
4986         printf("| %-35s | %-12s | %-4s |\n",
4987             "HGEMM WGMMMA", "SKIP", "SKIP");
4988         return;
4989     }
4990
4991     size_t size_a = (size_t)M * K * sizeof(half);
4992     size_t size_b = (size_t)K * N * sizeof(half);
4993     size_t size_c = (size_t)M * N * sizeof(half);
4994
4995     half *h_a = (half *)malloc(size_a);
4996     half *h_b = (half *)malloc(size_b);
4997     half *h_c_ref = (half *)malloc(size_c);
4998
4999     srand(42);
5000     for (int i = 0; i < M * K; i++)
5001         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5002     for (int i = 0; i < K * N; i++)
5003         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5004
5005     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
5006     size_t size_b_t = (size_t)N * K * sizeof(half);
5007     half *h_b_t = (half *)malloc(size_b_t);
5008     for (int n = 0; n < N; n++)
5009         for (int k = 0; k < K; k++)
5010             h_b_t[n * K + k] = h_b[k * N + n];
5011
5012     half *d_a, *d_b, *d_b_t, *d_c;
5013     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5014     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5015     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5016     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5017
5018     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5019     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5020         "wgmma H2D B (cuBLAS)");
5021     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5022         "wgmma H2D B_t (kernel)");
5023
5024     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5025     cublasHandle_t handle;
5026     cublasCreate(&handle);
5027     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5028     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5029         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5030         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5031     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5032         "wgmma D2H ref");
5033
5034     // Create TMA tensor maps for A and BT
5035     // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5036     // BT[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5037     CUtensorMap *tma_a =
5038         allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5039     CUtensorMap *tma_b =

```

```

5040     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5041
5042     // Launch WGMMMA kernel
5043     // BLOCK_SWIZZLE=false → 2D grid, no swizzle
5044     size_t smem_bytes =
5045         K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5046     cudaFuncSetAttribute(
5047         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
5048             false>,
5049         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5050
5051     dim3 block(NUM_THREADS);
5052     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5053     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
5054         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5055     check(cudaGetLastError(), "wgmma launch");
5056     check(cudaDeviceSynchronize(), "wgmma sync");
5057
5058     half *h_c = (half *)malloc(size_c);
5059     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5060
5061     // Verify
5062     float max_err = 0.0f;
5063     for (int i = 0; i < M * N; i++) {
5064         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5065         if (err > max_err)
5066             max_err = err;
5067     }
5068     printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
5069         max_err < 1.0f ? "PASS" : "FAIL");
5070
5071     free(h_a);
5072     free(h_b);
5073     free(h_b_t);
5074     free(h_c);
5075     free(h_c_ref);
5076     cudaFree(d_a);
5077     cudaFree(d_b);
5078     cudaFree(d_b_t);
5079     cudaFree(d_c);
5080     cudaFree(tma_a);
5081     cudaFree(tma_b);
5082     cublasDestroy(handle);
5083 }
5084 #endif /* NOTES_V2_ENABLE_WGMMMA */
5085
5086
5087 static void test_flash_attn(int seqlen, int head_dim) {
5088     // FlashAttention-2 with split-Q, MMA m16n8k16
5089     int B = 1, H = 8;
5090
5091     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5092
5093     srand(42);
5094     half *h_q = (half *)malloc(sz);
5095     half *h_k = (half *)malloc(sz);
5096     half *h_v = (half *)malloc(sz);
5097     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5098         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5099         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5100         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5101     }
5102

```

```

5103 // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
5104 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5105 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5106 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5107 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5108 int count = B * H * seqlen * head_dim;
5109 for (int i = 0; i < count; i++) {
5110     ref_q[i] = __half2float(h_q[i]);
5111     ref_k[i] = __half2float(h_k[i]);
5112     ref_v[i] = __half2float(h_v[i]);
5113 }
5114
5115 float scale = 1.0f / sqrtf((float)head_dim);
5116 for (int bi = 0; bi < B * H; bi++) {
5117     for (int qi = 0; qi < seqlen; qi++) {
5118         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
5119         float smax = -INFINITY;
5120         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5121         for (int kj = 0; kj < seqlen; kj++) {
5122             float s = 0.0f;
5123             for (int d = 0; d < head_dim; d++)
5124                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5125                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5126             S[kj] = s * scale;
5127             if (S[kj] > smax) smax = S[kj];
5128         }
5129         // softmax
5130         double sum_exp = 0.0;
5131         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5132         float inv_sum = 1.0f / (float)sum_exp;
5133         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5134         for (int d = 0; d < head_dim; d++) {
5135             double o_acc = 0.0;
5136             for (int kj = 0; kj < seqlen; kj++)
5137                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5138                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5139             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5140         }
5141         free(S);
5142     }
5143 }
5144
5145 half *d_q, *d_k, *d_v, *d_o;
5146 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5147 check(cudaMalloc(&d_k, sz), "fa alloc K");
5148 check(cudaMalloc(&d_v, sz), "fa alloc V");
5149 check(cudaMalloc(&d_o, sz), "fa alloc O");
5150 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5151 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5152 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5153
5154 // Template params for kHeadDim=64, kStage=2
5155 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
5156 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5157 constexpr int kMmaTileSeqLenQ = 4;
5158 constexpr int kMmaTileSeqLenK = 1;
5159 constexpr int kMmaTileSeqLenP = 4;
5160 constexpr int kMmaTileHeadDimV = 1;
5161 constexpr int kValTileSeqLenQ = 1;
5162 constexpr int kValTileSeqLenK = 8;
5163 constexpr int kValTileSeqLenP = 1;
5164 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5165

```

```

5166 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5167 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5168 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
5169                     kStageV * Bc * (kHeadDim + kPadV) +
5170                     Bc * (kHeadDim + kPadV)) * sizeof(half);
5171
5172 dim3 block(128);
5173 dim3 grid((seqlen + Br - 1) / Br, B * H);
5174
5175 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
5176 kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
5177 kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
5178 kStageV, kPadV>
5179 <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5180 check(cudaGetLastError(), "fa launch");
5181 check(cudaDeviceSynchronize(), "fa sync");
5182
5183 half *h_o = (half *)malloc(sz);
5184 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5185
5186 float max_err = 0.0f;
5187 for (int i = 0; i < count; i++) {
5188     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5189     if (err > max_err) max_err = err;
5190 }
5191 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
5192
5193 free(h_q); free(h_k); free(h_v); free(h_o);
5194 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5195 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5196 }
5197
5198
5199 int main(int argc, char *argv[]) {
5200 #if defined(NOTES_V2_ENABLE_WGMMMA)
5201     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
5202 #endif
5203     int M = 1024, N = 1024, K = 1024;
5204     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
5205
5206     printf("=== notes-v2.cu verification harness ===\n");
5207     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5208     printf("|-----|-----|-----|\n");
5209
5210     test_block_reduce(N);
5211     test_dot(N);
5212     test_relu(1024);
5213     test_elementwise(1024);
5214     test_histogram(1024);
5215     test_merge_attn_states(512, 16, 128);
5216     test_softmax(256);
5217     test_rms_norm(8, 128);
5218     test_layer_norm(8, 128);
5219     test_rope(8, 128);
5220     test_mat_transpose(256, 256);
5221     test_mat_transpose_padded(256, 256);
5222     test_sgemv(256, 128);
5223     test_sgemm(M, N, K);
5224     test_hgemm_mma(M, N, K);
5225     test_hgemm_swizzle(M, N, K);
5226 #if defined(NOTES_V2_ENABLE_CUTE)
5227     test_hgemm_cute(M, N, K);
5228 #endif

```

```

5229 #if (defined(NOTES_V2_ENABLE_WGMMMA))
5230     test_hgemm_wgmma(M, N, K);
5231 #endif
5232     test_flash_attn(1024, 64);
5233
5234     printf("=== All tests done ===\n");
5235     return 0;
5236 }
5237
5238 // =====
5239 // Quick build & run reference
5240 // =====
5241 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
5242 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
5243 // -I ../../third-party/cutlass/include \
5244 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
5245 //
5246 // # sm_89 单独编译 + 运行:
5247 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
5248 //
5249 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
5250 // 项目内置 third-party/cutlass) :
5251 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
5252 // -I ../../third-party/cutlass/include \
5253 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
5254 //
5255 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
5256 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5257 // -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
5258 //
5259 // # sm_90a + CuTe + WGMMMA:
5260 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5261 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMMA \
5262 // -I ../../third-party/cutlass/include \
5263 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```